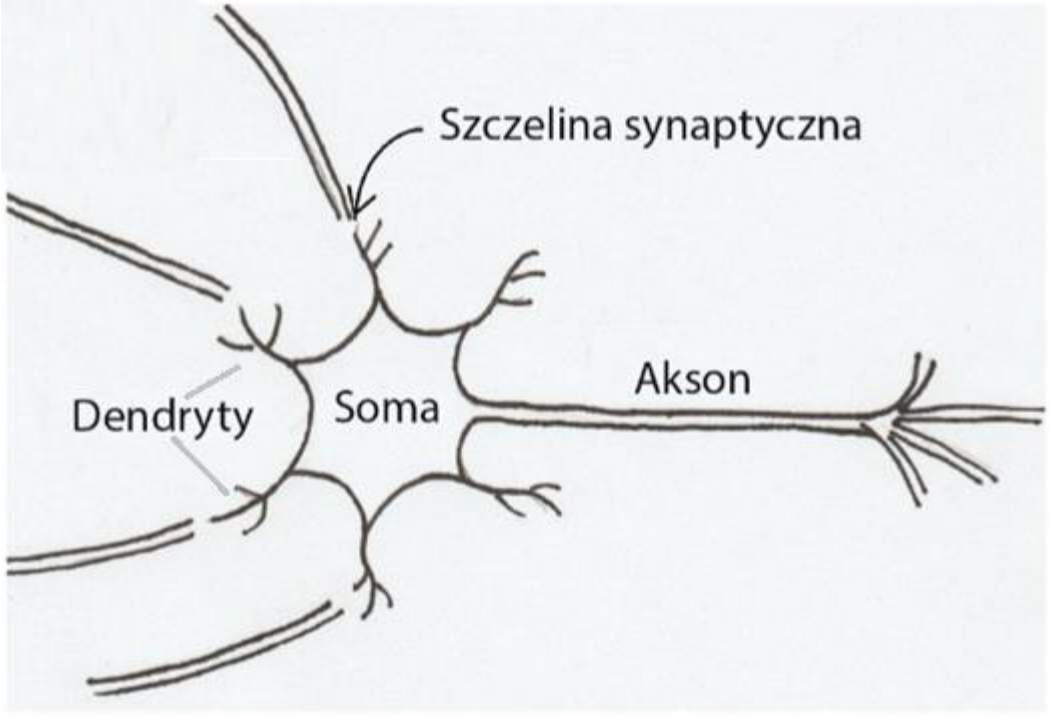
Podstawy sieci neuronowych

Dariusz Put

Plan prezentacji

1. Wprowadzenie
2. Wybrane architektury
3. Proces uczenia sieci neuronowych
4. Funkcje aktywacji
5. Przeuczenie i regularyzacja
6. Metody ewaluacji modeli

Neuron – jednostka przetwarzania, która przyjmuje wartość wejściową i, zgodnie z przyjętymi regułami, przekazuje obliczoną wartość na wyjściu.

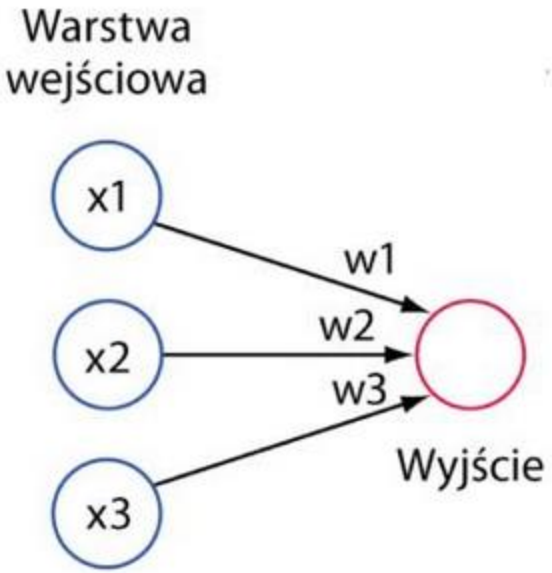


Dendryty – wejście
Soma – przetwarzanie
Akson – wyjście

Dendryty doprowadzają sygnał wejściowy z innych neuronów do somy. **Soma** (ciało neuronu) to miejsce, w którym dane wejściowe są przetwarzane. Jeśli sygnał wejściowy jest powyżej określonego progu, neuron „wyzwala się” i przekazuje pojedynczą daną wyjściową za pomocą sygnałów elektrycznych przez **aksony**.
Pomiędzy aksonami neuronu nadawczego a dendrytami neuronów odbierających znajduje się **szczelina synaptyczna**, która dostosowuje częstotliwości impulsów chemicznych.

Sieci neuronowe składają się z **neuronów**. Najprostszym przykładem sieci neuronowej jest **perceptron** (1957 rok). Jest to sieć składająca się wyłącznie z warstw wejścia i wyjścia. W przypadku klasyfikacji binarnych warstwa wyjścia ma jeden neuron.

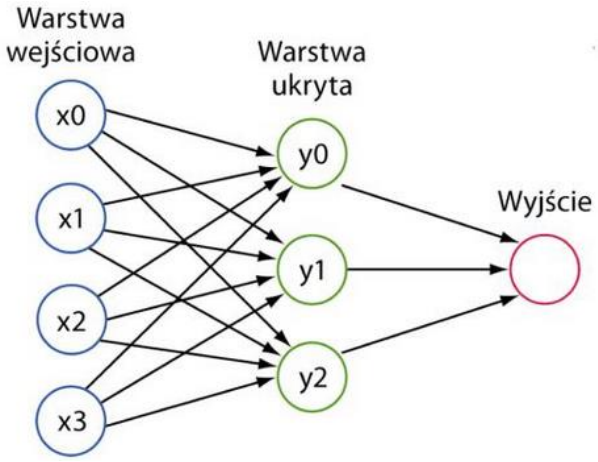
Na podstawie różnych danych wejściowych neuronu określamy **wartość aktywacji**. Jeśli wartość aktywacji, czyli wewnętrzny stan neuronu, przekroczy ustalony próg, neuron się uaktywni, w przeciwnym przypadku pozostanie uśpiony.



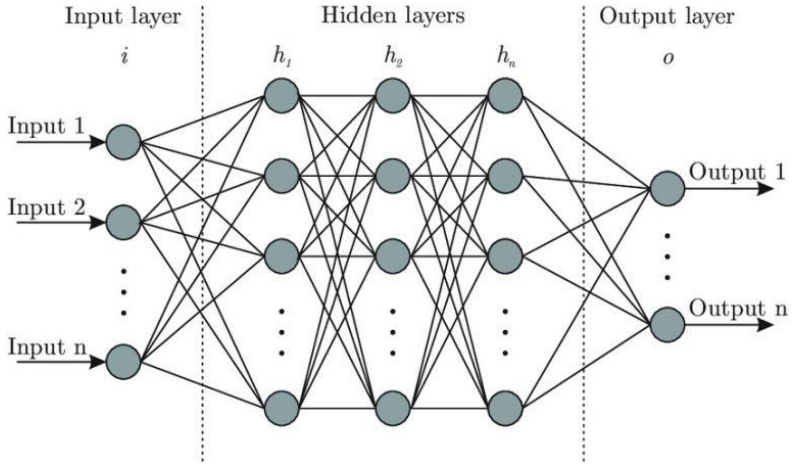
Prosty perceptron z trzema jednostkami wejściowymi (neuronami) i jedną jednostką wyjściową (neuronem)

Warstwa jest modułem przetwarzania danych, który przyjmuje dane wejściowe z jednego lub kilku tensorów i generuje dane wyjściowe w postaci tensora lub kilku tensorów.
Warstwy charakteryzowane są przez stan zwany **wagą**. Wagi tworzą **wiedzę** sieci.
Warstwa ukryta: odpowiedzialna za naukę i wykonywanie obliczeń. Nie można do niej bezpośrednio przekazywać informacji.

Sieci płytkie
mają maksymalnie jedną warstwę ukrytą.



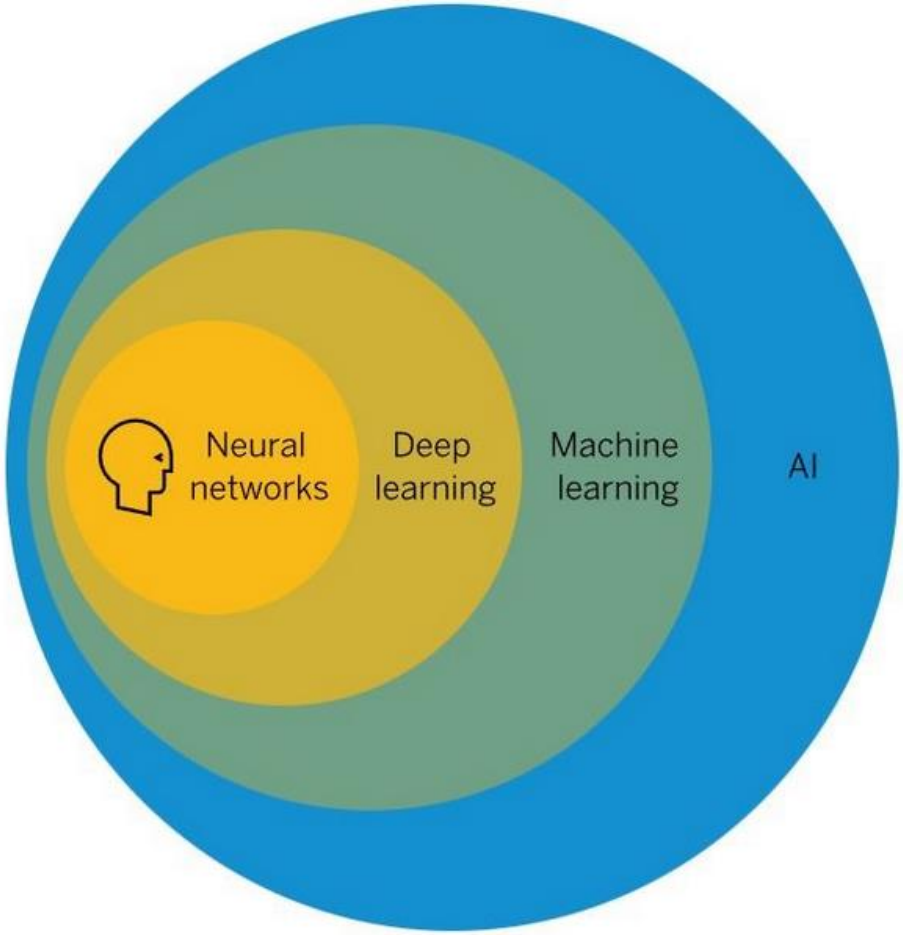
Sieci głębokie
zawierają wiele warstw ukrytych.



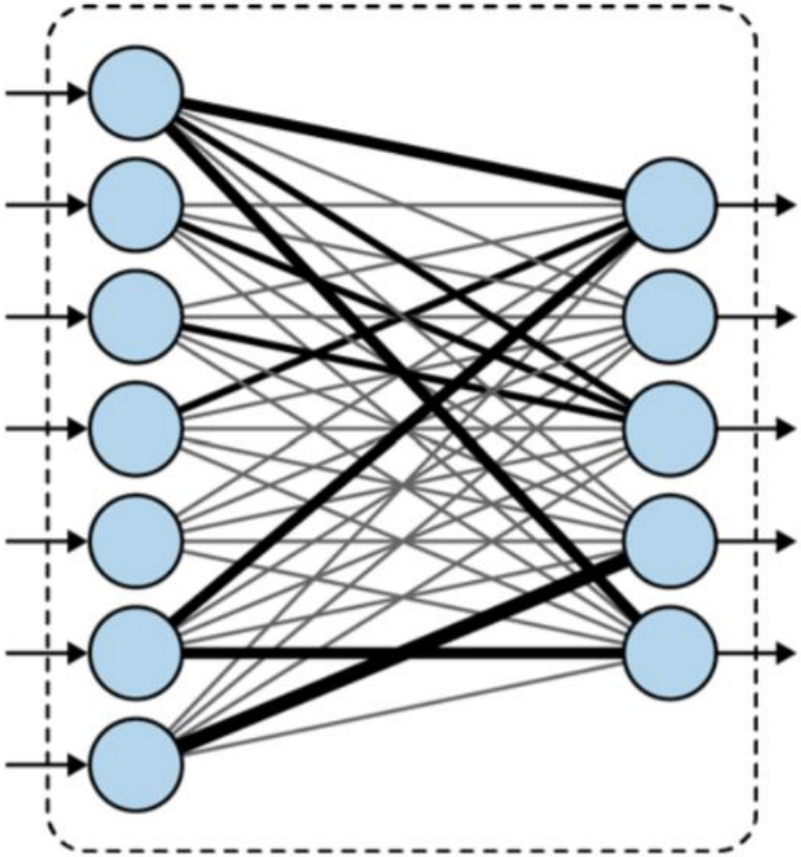
Zadaniem uczenia maszynowego jest ustalenie wartości wag (parametrów) poszczególnych warstw sieci.
Głębokie sieci neuronowe średniej wielkości zawierają nawet kilkadziesiąt milionów parametrów.

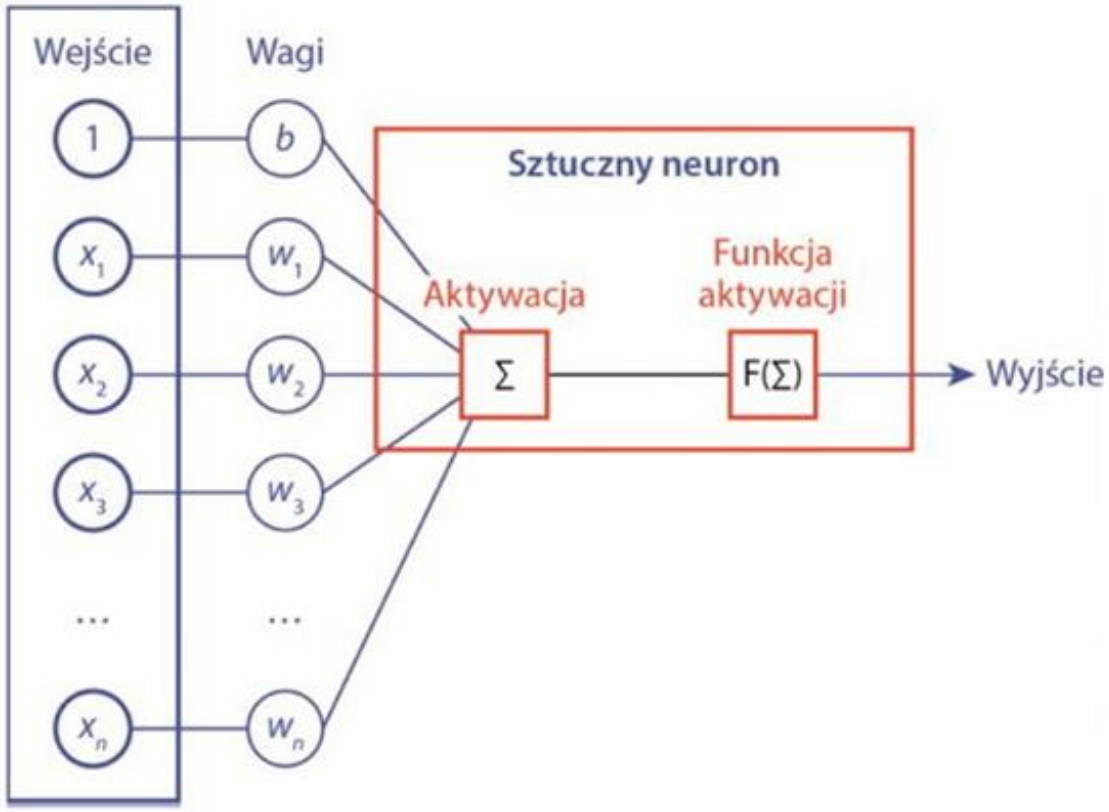
Uczenie głębokie – dziedzina uczenia maszynowego polegająca na tworzeniu wielu warstw (co najmniej dwóch) reprezentacji uczonych na podstawie danych treningowych. Te dodatkowe warstwy to tzw. **warstwy ukryte**. W uczeniu głębokim prawie zawsze wykorzystywane są **sieci neuronowe**.

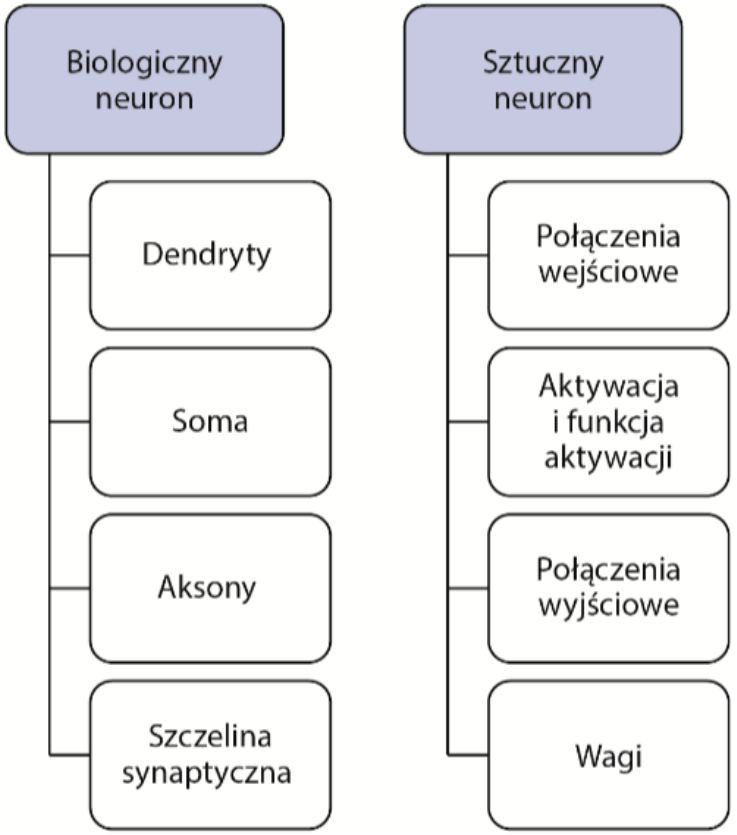
Głębia uczenia (**głębokość modelu**) – liczba warstw modelu.



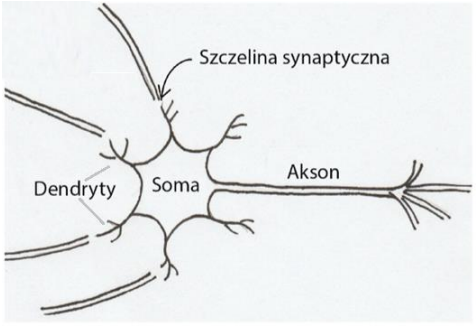
Warstwa w pełni połączona (*fully connected layer*, *dense layer*) – każdy neuron warstwy X jest połączony z każdym neuronem w warstwie X+1.

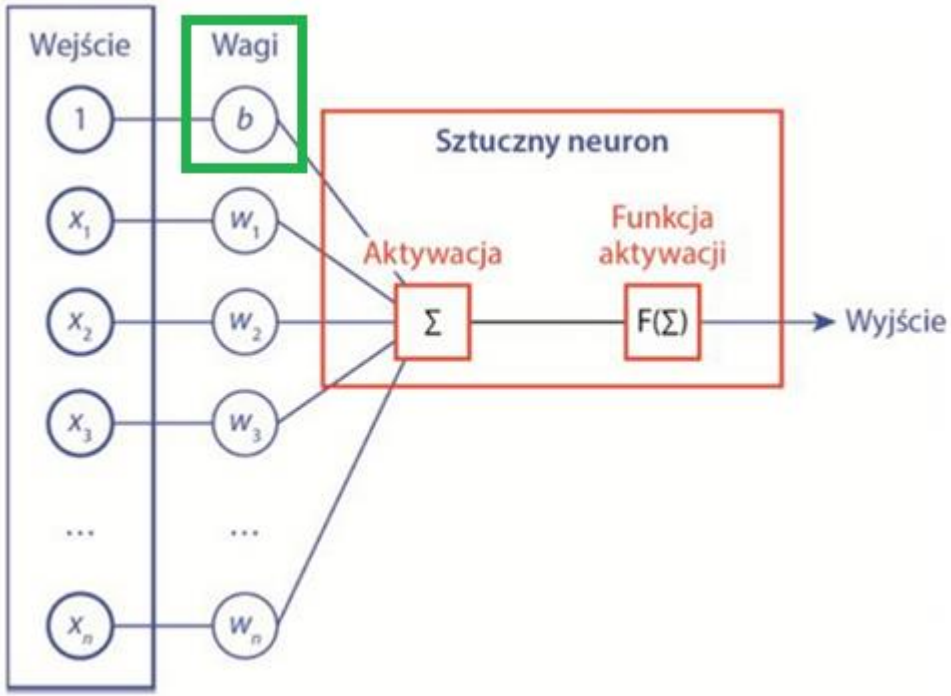






Sztuczny neuron (soma) otrzymuje wejście (**połączenia wejściowe**) i ustawia wewnętrzny stan, który wyzwala wyjście (**funkcja aktywacji**). **Sygnał wejściowy** jest przekazywany także do innych neuronów a jego intensywność jest dopasowywana za pomocą wag (**szczelina synaptyczna**).





Każda warstwa, poza wyjściową, jest doposażona w stałą wartość przesunięcia, tzw. **wartość progową** (obciążenie, ang. ***b-bias***). Jest to dodatkowy neuron zwykle przechowujący wartość 1. Jego waga również jest modyfikowana w trakcie procesu uczenia. Bias jest niezależnym czynnikiem, który pozwala lepiej dopasowywać się sieci do oczekiwanego rezultatu.

Własności sieci neuronowych:

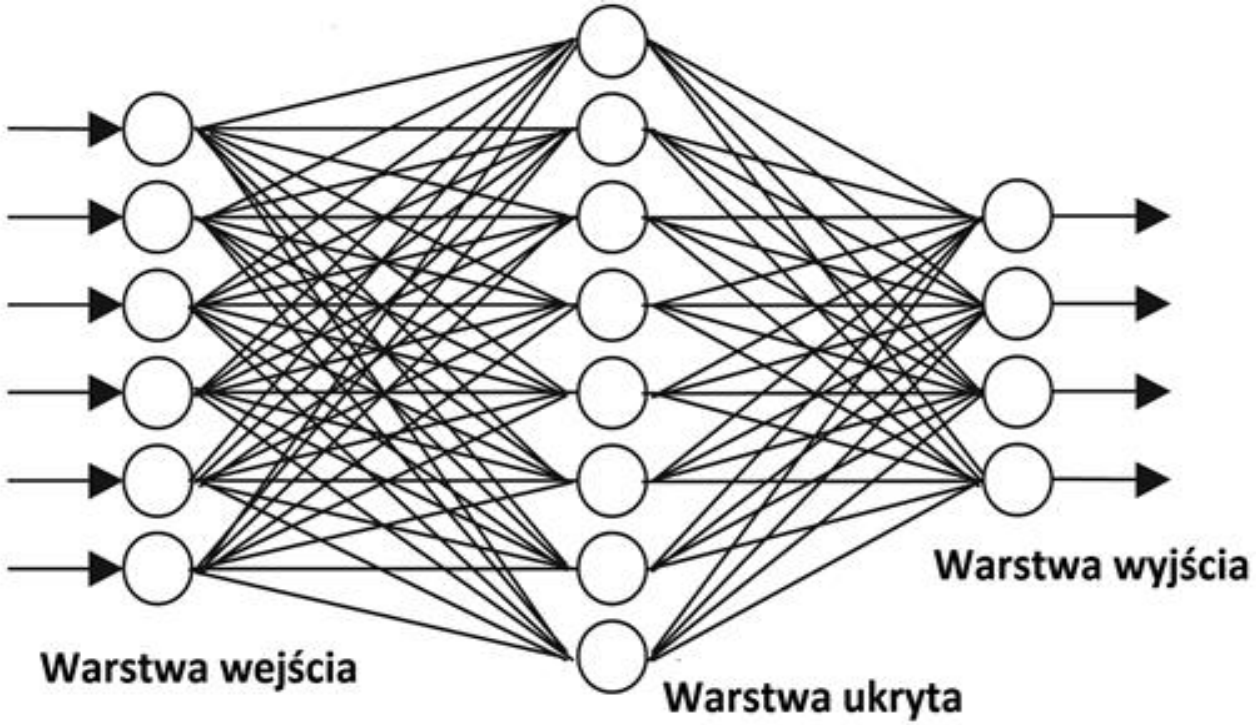
- **architektura –** sposób połączeń pomiędzy neuronami (np. ze sprzężeniem zwrotnym, bez sprzężenia zwrotnego), liczba warstw (jedno lub wielowarstwowe) i neuronów w warstwie,
- **uczenie** – opisuje proces powszechnie definiowany jako trening. Może być prowadzony za pomocą różnych technik, np. metodą propagacji wstecznej. Identyfikuje sposób, w jaki określane są wagi neuronów,
- **funkcja aktywacji** – opisuje, jak działa neuron (stochastycznie, liniowo) oraz w jakich warunkach zostanie uaktywniony a także dane, jakie przekaże do sąsiednich neuronów. Jest obliczana na podstawie wyjść wszystkich neuronów wejściowych.

Wybrane architektury sieci neuronowych

1. Sieci jednokierunkowe

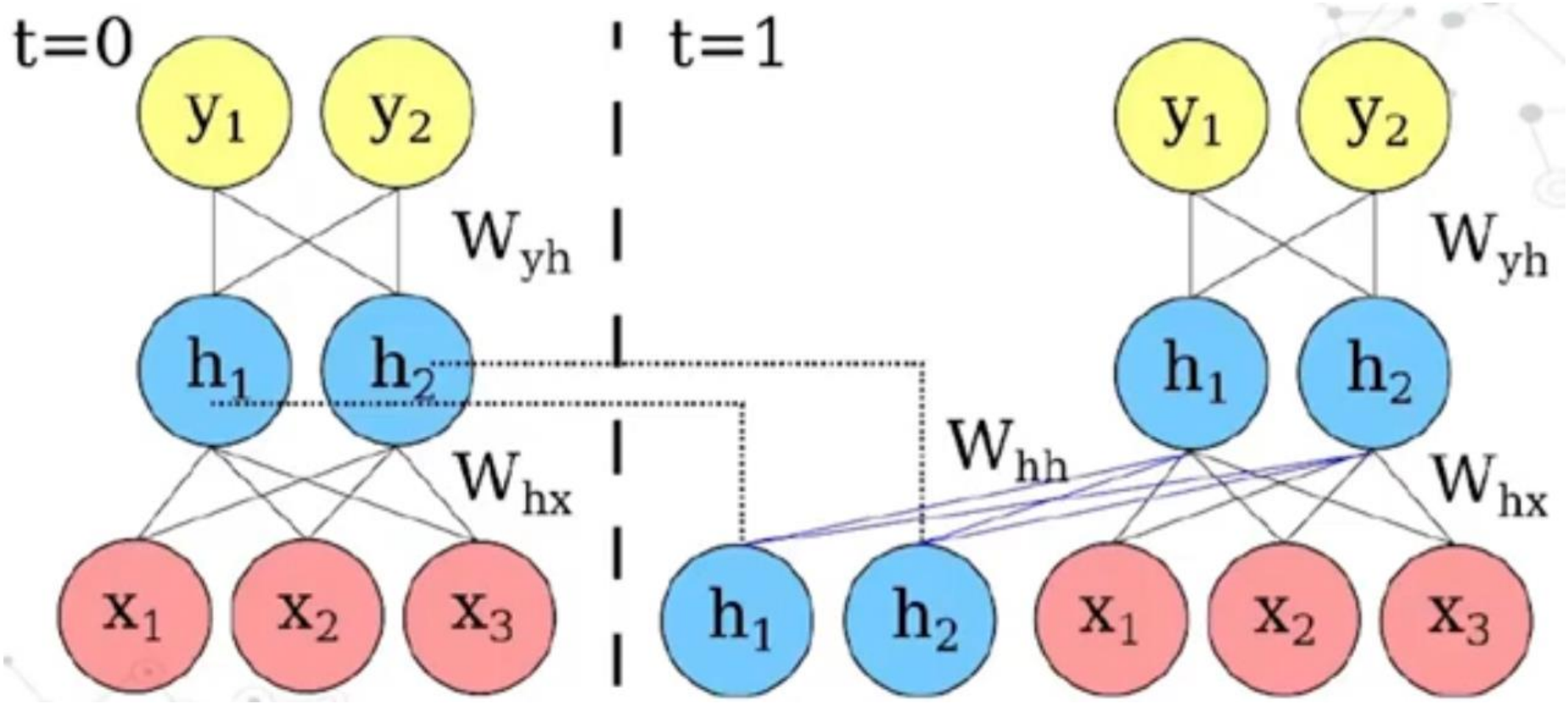
Są to sieci, które charakteryzują się budową warstwową oraz tym, że neurony są połączone tylko z tymi z sąsiednich warstw. Nie występuje w nich sprzężenie zwrotne, a informacja przepływa w jednym kierunku.

Wyróżniamy sieci jedno-, dwu- bądź wielowarstwowe. Te ostatnie mogą rozwiązywać dużo szerszą klasę problemów, niż te zbudowane z pojedynczej warstwy neuronów.



2. Sieci rekurencyjne (RNN – *Recursive Neural Networks*)

To rodzaj sieci, w których neurony są połączone ze sobą za pomocą struktury grafu z cyklami. Posiadają własną pamięć, która zapamiętuje informacje, które wcześniej zostały obliczone. Do ich głównych przedstawicieli należy sieć Hopfielda (inaczej sieć asocjacyjna) oraz tak zwana maszyna Boltzmanna, która jest modyfikacją sieci asocjacyjnej i pozwala na uczenie neuronów ukrytych. Oba rodzaje sieci rekurencyjnych znajdują zastosowanie przy rozpoznawaniu mowy lub obrazów, rozwiazywaniu problemów minimalizacji czy przetwarzaniu danych sekwencyjnych.



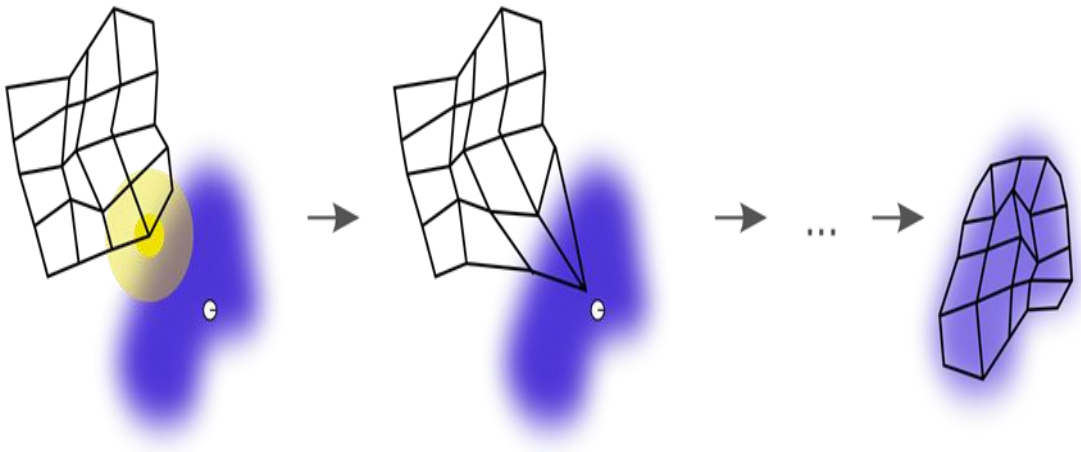
W klasycznych sieciach RNN występuje problem tzw. zanikającego gradientu (ang. *vanishing gradient problem*), gdyż podczas uczenia wagi muszą być aktualizowane nie tylko w dół, ale także w poprzek sekwencji. Wraz ze wzrostem długości sekwencji aktualizacja wag jest na granicy błędu statystycznego, przez co uczenie staje się nieefektywne. Problem ten powoduje, że klasyczna sieć RNN nie jest w stanie zapamiętać zbyt długich sekwencji (np. wyrazów).

3. Rekurencyjne sieci LSTM (*Long Short-Term Memory* – Bengio 1994)
Sieć LSTM stanowi rozwiązanie problemu zanikającego gradientu. Sieć uczy się, co przechowywać w stanie długoterminowym. Wykorzystuje informacje z wielu poprzednich kroków. Sieć decyduje, jak dużo informacji z przeszłości należy przechowywać oraz ile informacji z teraźniejszości zastosować.
Sieci tego typu znajdują zastosowanie w modelowaniu językowym (np. generowaniu tekstu), gdzie znaczenie zdania złożonego może zależeć od relacji między odległymi słowami.

Inną siecią mającą podobne działanie, ale o mniejszej złożoności obliczeniowej, jest sieć **GRU** (Gatet Recurrent Unit).

4. Samoorganizujące się mapy (sieci Kohonena)

Są to sieci, z którymi są sprzężone punkty na dowolnej n-wymiarowej przestrzeni. Uczenie ich odbywa się w trybie bez nauczyciela/nadzorcy i polega na zmianie współrzędnych neuronów tak, by dążyły do wzorca stworzonego na podstawie danych wejściowych i odwzorowały ich strukturę. Tego rodzaju sieci wykorzystywane są w klasyfikacji wzorców.



5. Perceptrony wielowarstwowe (MLP – Multilayer Perceptrons)
Sieci neuronowe z wieloma warstwami ukrytymi, bez sprzężenia zwrotnego. To jedno z pierwszych rozwiązań stosowanych w uczeniu głębokim.

6. Maszyny Boltzmanna (BM – Boltzmann Machines)
Sieci stochastyczne, symetryczne, z połączeniami wewnątrz warstwy.

7. Ograniczone maszyny Boltzmanna (RBM – Restricted Boltzmann Machines)
Składają się z połączeń symetrycznych między dwoma warstwami – jedną widoczną i jedną ukrytą, ale w odróżnieniu od ogólnych maszyn Boltzmanna neurony nie mają połączeń wewnątrz warstwy. Mogą być ułożone w stos tworząc w ten sposób sieć DBN (*Deep Belief Network*). Wykorzystywane w uczeniu nienadzorowanym.

8. Skierowane grafy acykliczne
(DAG – *Directed Acyclic Graph*) – graf skierowany, który nie posiada cykli skierowanych.

9. Sieci „głębokiej wiary” (DBN – *Deep Belief Networks*)

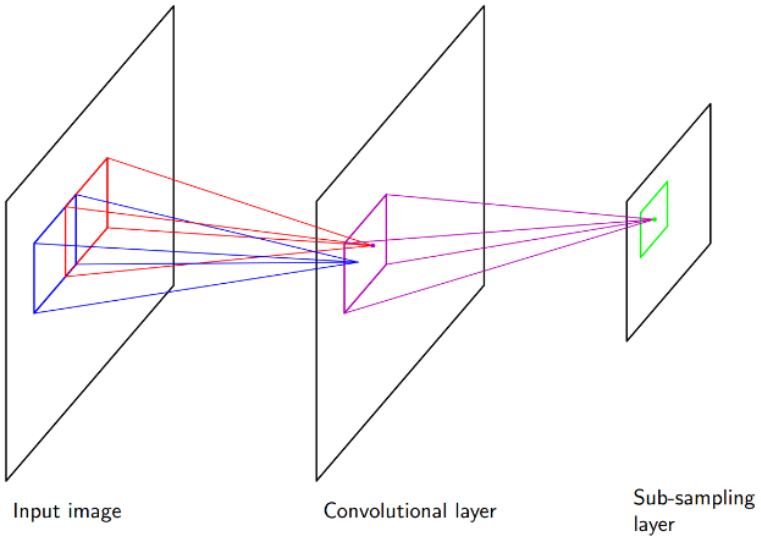
Górne warstwy są ze sobą połączone symetrycznie, dolne warstwy odbierają od warstw ponad nimi przetworzone informacje za pośrednictwem połączeń jednokierunkowych.

10. Autoenkodery (*Autoencoder*)

To jedna z metod redukcji wymiarów oparta o rzutowanie jednego wymiaru na inny. Poszukują reprezentacji zbioru danych poprzez uczenie sieci ignorowania szumu. Wykorzystywane w uczeniu nienadzorowanym.

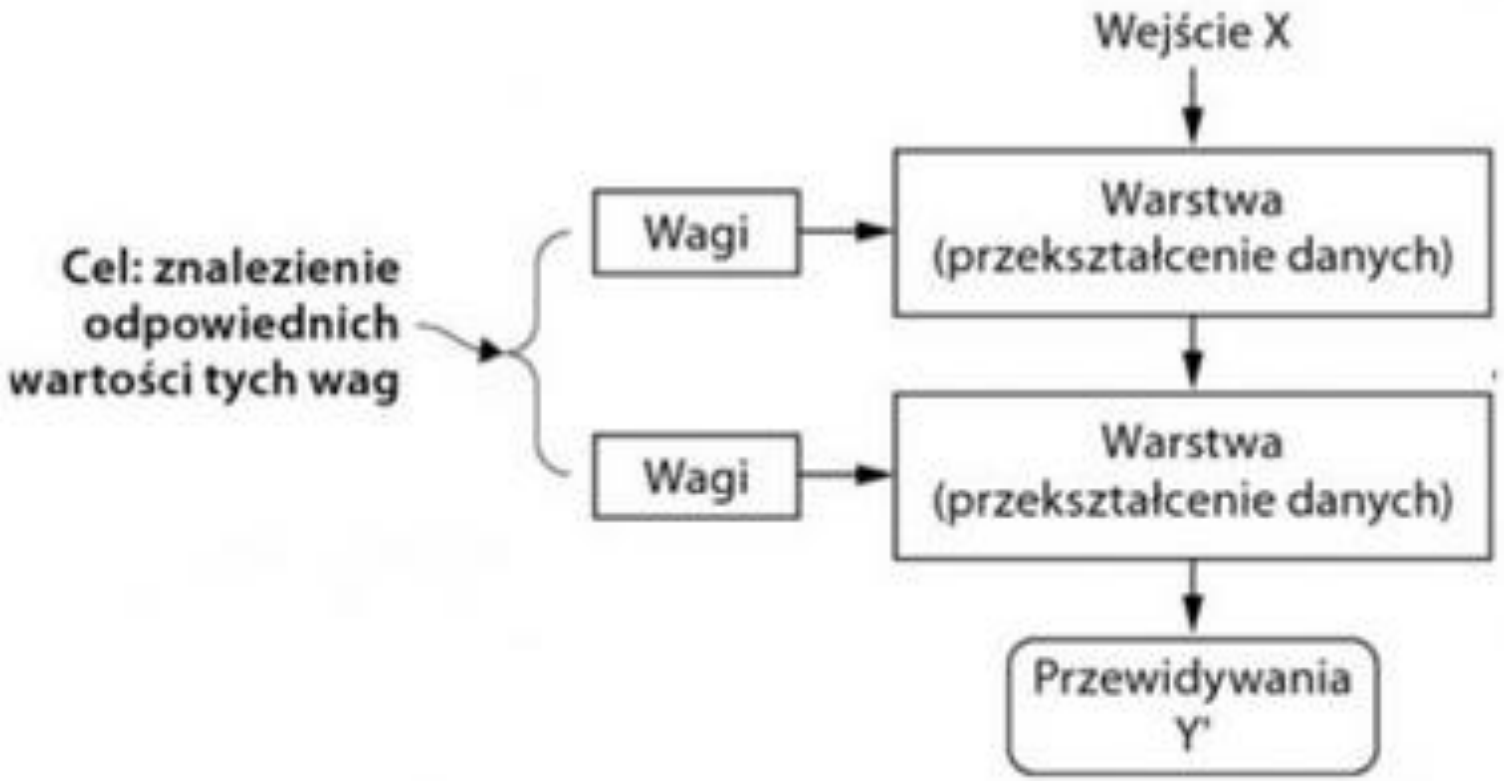
11. Konwolucyjne (splotowe) sieci neuronowe (CNN – *Convolutional Neural Networks*)

Warstwy stosują filtry do obrazu lub dźwięku wejściowego, przesuwając ten filtr przez cały sygnał wejściowy w celu uzyskania dwuwymiarowej mapy aktywacji.



Proces uczenia sieci neuronowych

Uczenie (trenowanie) polega na poszukiwaniu zbioru wartości wag wszystkich warstw sieci, aby sieć poprawnie mapowała dane wejściowe na dane wyjściowe.



W celu kontrolowania działania wyjść sieci neuronowej należy dysponować sposobem pomiaru tego, jak aktualnie generowane dane odbiegają od oczekiwań. Służy do tego **funkcja straty** sieci zwana też **funkcją celu**.
Funkcja straty (celu) to wartość, którą sieć próbuje minimalizować podczas trenowania. Oblicza, jak bardzo wartości obliczone przez sieć odbiegają od oczekiwanych i w ten sposób uzyskiwana jest odpowiedź na pytanie, jaka jest odległość rozkładu przewidywanego od rzeczywistego. Jest miarą sukcesu wykonywanego zadania. Jest używana w charakterze informacji zwrotnej umożliwiającej dostrojenie wartości wag.

Funkcje straty są wykorzystywane w **metodzie spadku wzdłuż gradientu**, który jest iteracyjnym algorytmem przesuwania parametrów (współczynników) w kierunku wartości optymalnych (jak najbardziej zbliżonych do rzeczywistych).

Funkcja celu musi być poprawnie dobrana, gdyż sieć będzie robiła wszystko aby minimalizować straty. Przykład: „maksymalizuj średni dobrobyt wszystkich ludzi”. Aby osiągnąć cel funkcja może dążyć do zabicia wszystkich ludzi oprócz kilku najbogatszych.

Nie istnieje jedna uniwersalna funkcja straty, która sprawdzałaby się w każdym rodzaju problemu. Musi być ona dobrana na podstawie analizy typu problemu i zestawu danych wejściowych.

Istnieją wskazówki umożliwiające **wybór właściwej funkcji straty**. Np.:

- do zadania klasyfikacji najczęściej korzysta się z entropii krzyżowej, w zadaniach klasyfikacji binarnej jest to **binarna entropia krzyżowa** (binary_crossentropy), a w klasyfikacji wieloklasowej **kategoryzacyjna entropia krzyżowa** (categorical_crossentropy). Inne funkcje straty wykorzystywane w zadaniach klasyfikacji to funkcja logarytmiczna, rozbieżność Kullbacka-Leiblera czy funkcja wykładnicza,
- problem regresji jest rozwiązywany za pomocą **średniego błędu kwadratowego** (mse – *mean squared error*), średniego błędu bezwzględnego (mae – *mean absolute error*), funkcji Hubera (odmiana mae) czy logarytmicznego cosinusa hiperbolicznego.

Funkcje celu należy dobierać samodzielnie w zasadzie tylko podczas pracy z nowym problemem badawczym.

Metoda entropii krzyżowej (algorytm uczenia przez wzmacnianie) –
to technika rozwiązywania zadań optymalizacji, czyli takich, które polegają na znalezieniu najlepszych parametrów pozwalających zminimalizować lub zmaksymalizować daną funkcję. W metodzie tej wykorzystywane są informacje zwrotne w celu znalezienia optymalnego rozwiązania problemu.

Przykład:
Jak dobrać parametry elementów samolotu (wielkość skrzydeł, kąt ich nachylenia, waga samolotu, itd.), żeby przeleciał jak najdłuższą trasę w określonych warunkach.

Działanie funkcji straty na przykładzie entropii krzyżowej.

Entropia krzyżowa jest obliczana według wzoru:

$$H(p,q) = -\sum_{x} p(x)\ \log q(x)$$

gdzie:
p(x) prawdopodobieństwo rzeczywiste
q(x) prawdopodobieństwo obliczone

Zwykle jako podstawa logarytmu wykorzystywana jest liczba 2.
Jeśli liczba logarytmowana jest równa zero [q(x)=0], przyjmuje się, że logarytm wynosi 0.

Entropia krzyżowa – przykład:

Zadanie polega na rozpoznaniu, czy na zdjęciu jest kot, pies, słoń czy zebra. Klasa (czyli zbiór etykiet) jest zbiorem czteroelementowym: K, P, S, Z.

Rzeczywista (jednorazowa) dystrybucja (rozkład) prawdopodobieństwa to dystrybucja prawdziwa. Jeśli na zdjęciu jest słoń, to rzeczywisty rozkład prawdopodobieństwa jest następujący:

K	P	S	Z
0	0	1	0

Załóżmy, że w procesie uczenia maszynowego uzyskano następujący rozkład prawdopodobieństwa:

K	P	S	Z
0.03	0.05	0.87	0.05

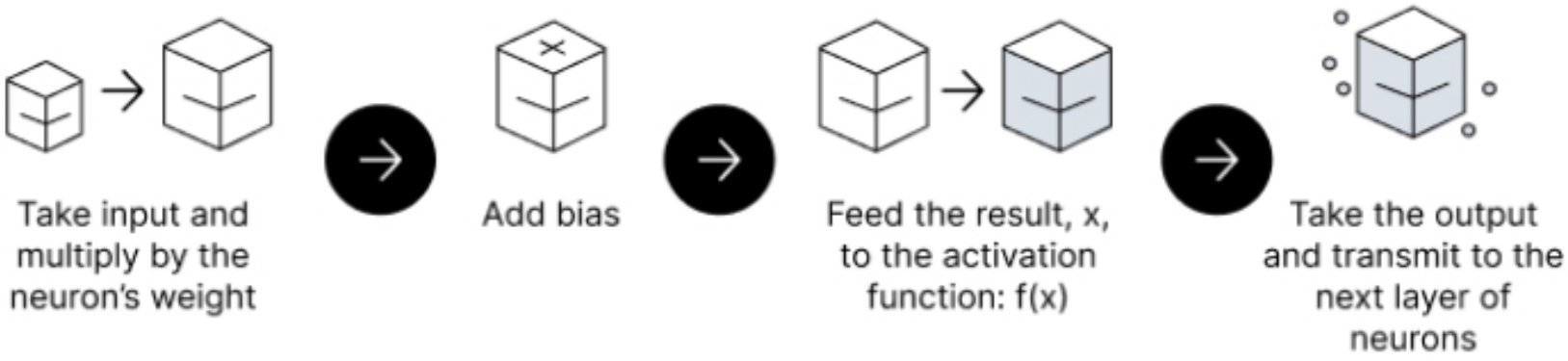
$$H(p,q) = -\sum_{x} p(x) \log q(x)$$

Czyli strata H(p,q) jest równa:

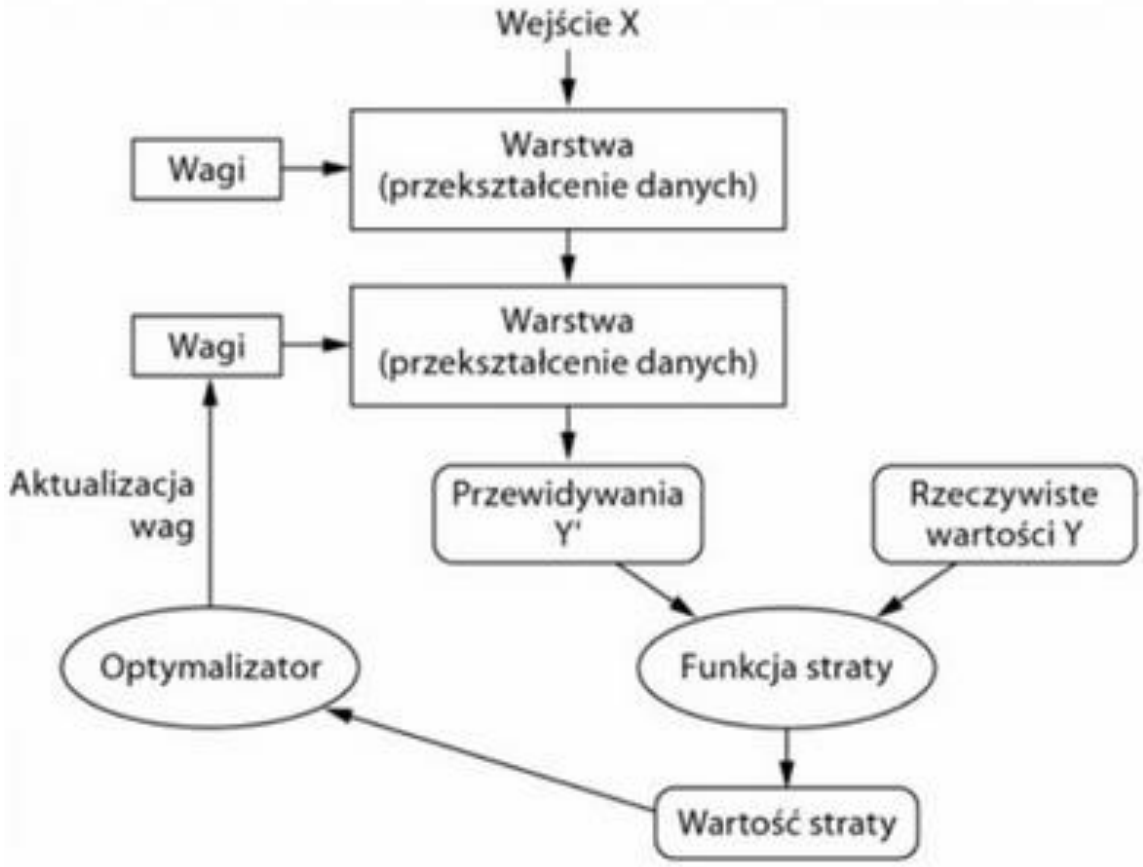
$-(0*\log_2 0{,}03 + 0*\log_2 0{,}05 + \mathbf{1*\log_2 0{,}87} + 0*\log_2 0{,}05) = -1*\log_2 0{,}87 = \mathbf{0{,}20}$

W procesie uczenia sieci wykorzystywana jest tzw. **propagacja w przód** (ang. *feedforward propagation*) – przepływ informacji odbywa się do przodu. Dane wejściowe są wykorzystywane do obliczenia wartości pewnej funkcji pośredniej w warstwie ukrytej, która jest następnie wykorzystywana do obliczenia danych wyjściowych.

W propagacji w przód funkcja aktywacji jest matematyczną „bramką” pomiędzy wejściem zasilającym aktualny neuron a jego wyjściem, które trafia do następnej warstwy.



Wynik obliczony przez funkcję straty stanowi informację zwrotną umożliwiającą dostrojenie wartości wag w taki sposób, aby zmniejszyć wartość błędu. Dostrajanie wag jest wykonywane przez **optymalizator** implementujący algorytm **propagacji wstecznej**.



Początkowo wagom przypisuje się losowe wartości, więc strata sieci jest duża. Ale po dostrojeniu podczas kolejnej iteracji algorytmu wartość straty maleje. Jest to tzw. **pętla trenowania**, która polega na wielokrotnej iteracji algorytmu propagacji wstecznej w celu dostrajania wag i zmniejszania funkcji straty. Sieć z minimalną stratą generuje dane wyjściowe zbliżone do oczekiwanych. Taką sieć nazywamy **wytrenowaną**.

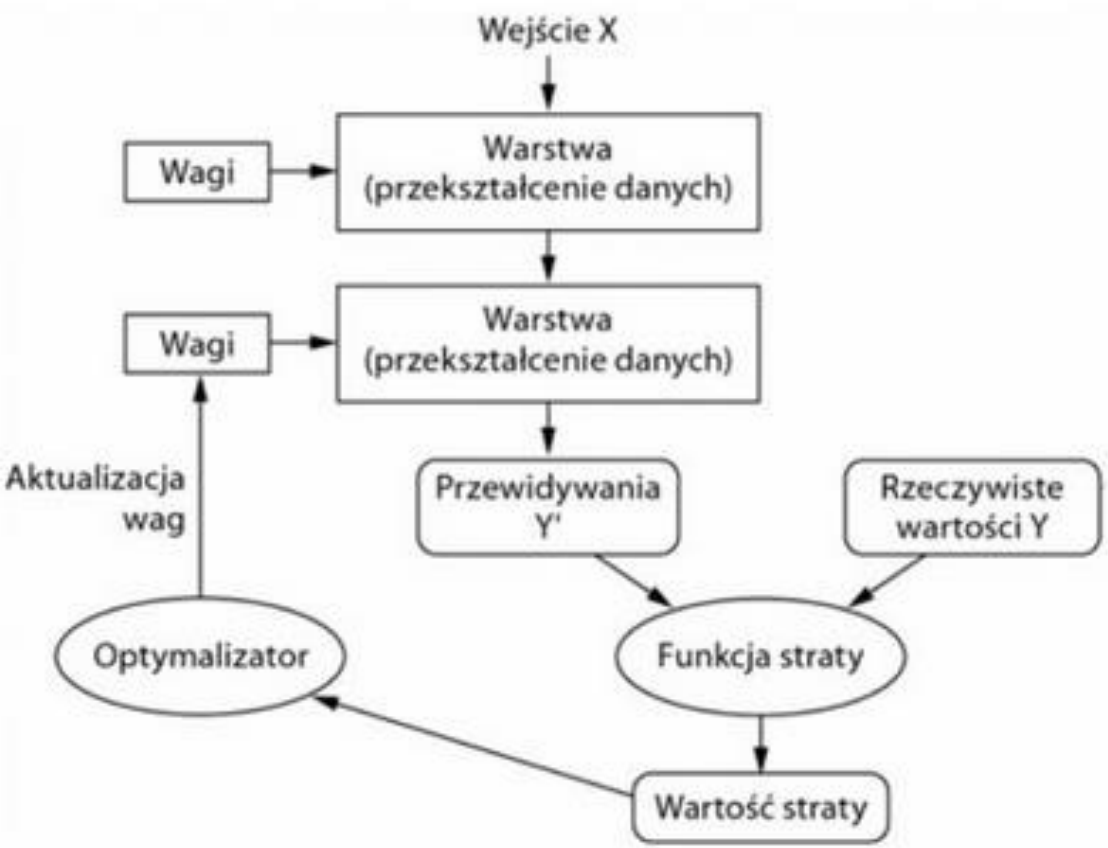
Algorytm wstecznej propagacji błędów (ang. *back propation algorithm*)
jest najbardziej popularnym algorytmem wykorzystywanym do uczenia sieci wielowarstwowych.

Działanie tego algorytmu polega na wielokrotnym dopasowywaniu wag neuronów w celu zminimalizowania różnicy między wektorem wyjściowym sieci a rzeczywistym wektorem wyjściowym.

Jego działanie oparte jest na tzw. regule delta: wyznaczamy funkcję błędu dla sieci, a następnie dążymy do znalezienia minimum tej funkcji względem wektora w. Uczenie składa się z dwóch naprzemiennych faz:

1. Fazy propagacji sygnału od wejść (wektora x) do wyjścia.
2. Fazy wstecznej propagacji błędu od wyjścia y w kierunku wejść sieci.

Aby obliczyć wartość, o jaką dany współczynnik powinien zostać pomniejszony lub powiększony oblicza się **gradient** straty z uwzględnieniem współczynników sieci.
W celu zmniejszenia straty podczas kolejnej iteracji, należy przesunąć współczynniki w kierunku **przeciwnym do kierunku gradientu**.
Dlaczego?…



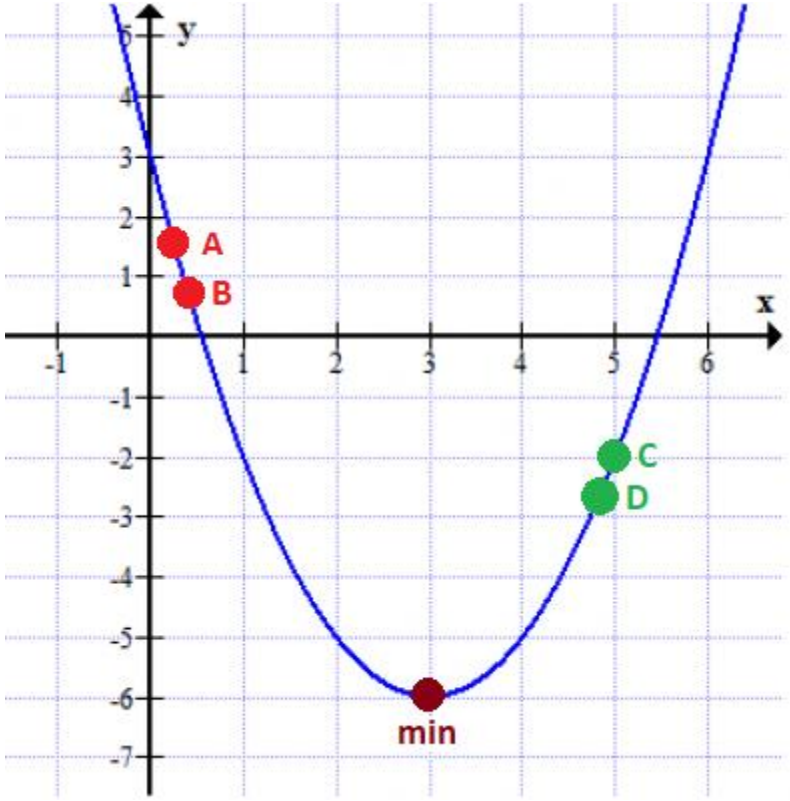
Załóżmy, że funkcja straty jest funkcją kwadratową:

$y=x^2 – 6x + 3$

Dążymy do minimalizacji straty, więc szukamy minimum funkcji (**min**).

Załóżmy, że po wykonaniu pierwszej iteracji (po pierwszym przejściu przez sieć) wartość funkcji straty znajduje się w punkcie A.

Powstaje pytanie, w którym kierunku należy podążać aby w następnym kroku zmniejszyć stratę i znaleźć się np. w punkcie B?



Wyznaczamy pochodną funkcji y:

$y' = 2x – 6$

Liczymy pochodną w punkcie A (dla x=0,25):

2*0,25–6 = –5,5

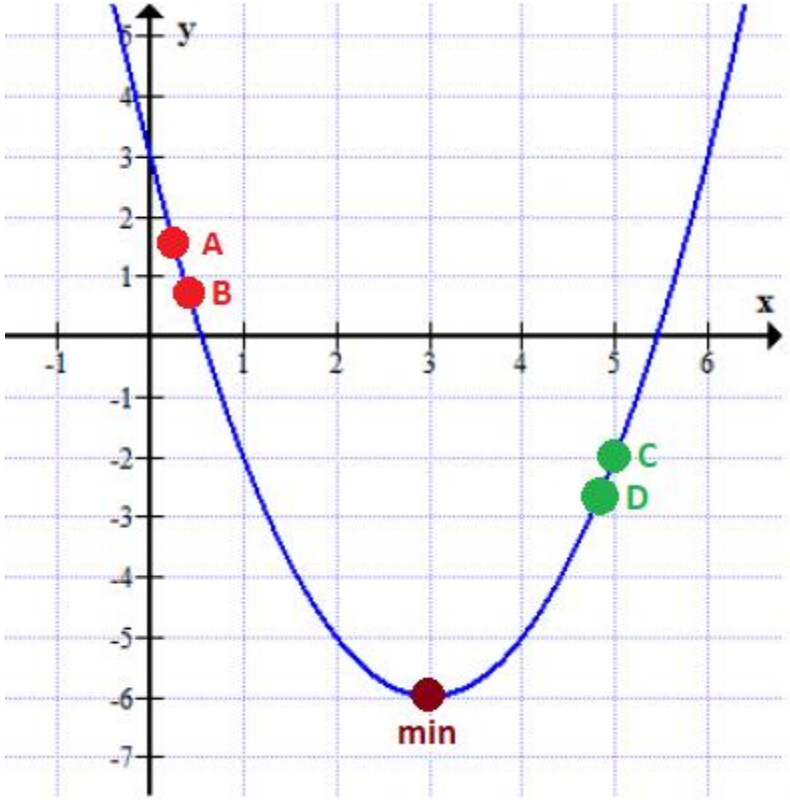
Pochodna jest ujemna, więc poruszamy się w przeciwnym kierunku minimalnie **zwiększając** wartość x – po następnej iteracji znajdziemy się w punkcie B, czyli bliżej minimum.

$y=x^2 - 6x + 3$

$y' = 2x - 6$

Załóżmy, że po pierwszej iteracji znaleźliśmy się w punkcie C. Liczymy pochodną w punkcie C (dla x=5):
2*5–6 = 4
Pochodna jest dodatnia, więc poruszamy się w przeciwnym kierunku minimalnie zmniejszając wartość x – po następnej iteracji znajdziemy się w punkcie C, czyli bliżej minimum.

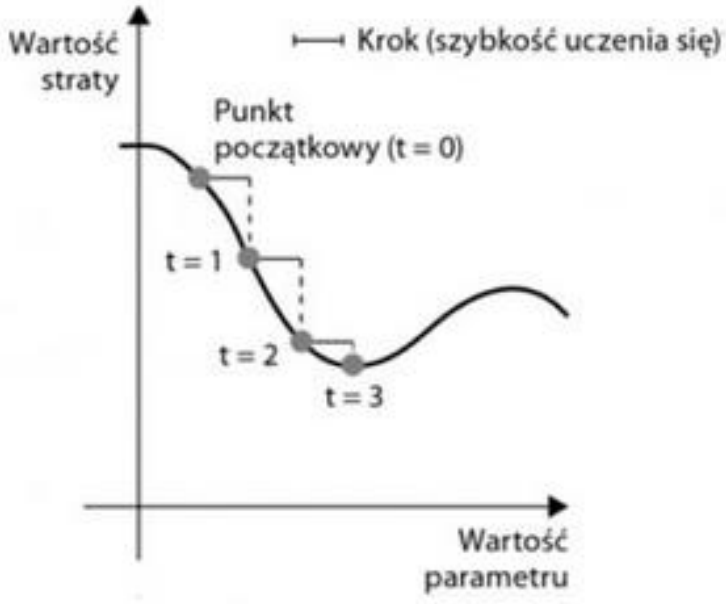
Dysponując pochodną możemy znaleźć minimum funkcji, gdyż w minimum pochodna jest równa 0.



Gradient jest pochodną funkcji tensorowej. Jest to uogólnienie koncepcji pochodnych funkcji przyjmujących wiele argumentów, tj. funkcji przetwarzających tensory.

Aby znaleźć minimum funkcji straty należy postępować według następującego algorytmu – jest to tzw. **algorytm stochastycznego spadku wzdłuż gradientu mini-batch (mini-batch SGD)**:

1. Wygeneruj wsad składający się z próbek treningowych (x) i odpowiadających im wartości docelowych (y).
2. Uruchom sieć na próbkach (x) w celu uzyskania przewidywanych wartości (y_pred).
3. Oblicz stratę sieci i zmierz różnice między wartościami docelowymi (y) a przewidywanymi (y_pred).
4. Oblicz gradient straty w odniesieniu do parametrów sieci.
5. Przesuń wartości parametrów w kierunku przeciwnym do gradientu, co powinno nieco zredukować stratę na analizowanej próbce danych treningowych.



Stosowanie algorytmu mini-batch SGD wymaga używania małego kroku (aby nie minąć minimum).

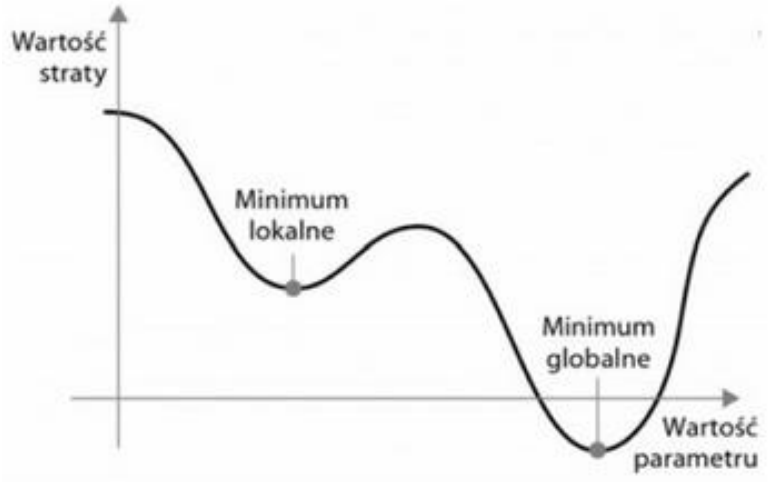
Gradient ustala tylko, w którym kierunku powinny zostać zmienione wagi. Nie ustala on, jak szybko trzeba się poruszać w obrębie funkcji, żeby dotrzeć do jej minimum. Jeżeli wartość **współczynnika uczenia** zostanie ustalona przez badacza jako za duża, minimum zostanie przestrzelone. Z tego powodu, sieć będzie uczyć się długo i będzie oscylowała daleko od minimum. Z kolei, jeżeli wartość ta zostanie wybrana jako za mała, sieć będzie uczyła się zbyt wolno.

Nie jest możliwe wyznaczenie współczynnika uczenia za pomocą metod analitycznych. Wstępnie ustalany jest z reguły jako wartość w przedziale od 0.1 do 0.01, po czym jest on dostosowywany do modelu. Jedynym sposobem na to dostosowywanie jest metoda prób i błędów.





Kolejnym problemem jest wpadnięcie w lokalne minimum:



W celu rozwiązania tego problemu wykorzystano pojęcie pędu. Proces optymalizacji można porównać do staczającej się kuli. Jeśli kula będzie miała wystarczająco duży pęd, nie utknie w minimum lokalnym, tylko potoczy się dalej. Pęd jest implementowany poprzez wykonywanie kolejnych kroków nie tylko na podstawie aktualnej wartości nachylenia (aktualnego przyspieszenia) ale także aktualnej prędkości wynikającej z poprzedniego przyspieszenia. W praktyce oznacza to aktualizację parametrów nie tylko na podstawie bieżącej wartości gradientu ale na podstawie wcześniejszych zmian tych parametrów.

Istnieją wersje algorytmu SGD, które podczas obliczania kolejnych wag biorą pod uwagę ich wcześniejsze aktualizacje oraz uwzględniają pęd. Istnieją też inne algorytmy oparte na tych rozwiązaniach: AdaGrad i RMSProp. Takie rozwiązania nazywa się **metodami optymalizacji** lub **optymalizatorami**.

Optymalizator – narzędzie służące do dostrajania wartości wag na podstawie funkcji straty. Implementuje on określony wariant algorytmu spadku wzdłuż gradientu.

Koncepcja gradientu prostego:
iteracyjny algorytm optymalizacji w celu znalezienia minimum funkcji. Jest to metoda kosztowna, ponieważ robimy tylko jeden krok (raz aktualizujemy wagi) po pojedynczym przejściu przez cały zbiór danych treningowych.
W metodzie **stochastycznego spadku wzdłuż gradientu** (SGD – *Stochastic Gradient Descent*), aktualizacja wag następuje po każdym przykładzie treningowym, bez czekania aż cały zestaw danych przejdzie przez algorytm.

Adam (ang. *Adaptive Moment Estimation*) jest wariantem SGD. W optymalizacji Adam aktualizujemy wskaźnik uczenia (wagę) na podstawie średniej i wariancji poprzednich gradientów tego parametru. Optymalizacja ta okazała się niezwykle skuteczna i szybka w rozwiązywaniu wielu problemów uczenia głębokiego. Często stosowany i jeden z najefektywniejszych.

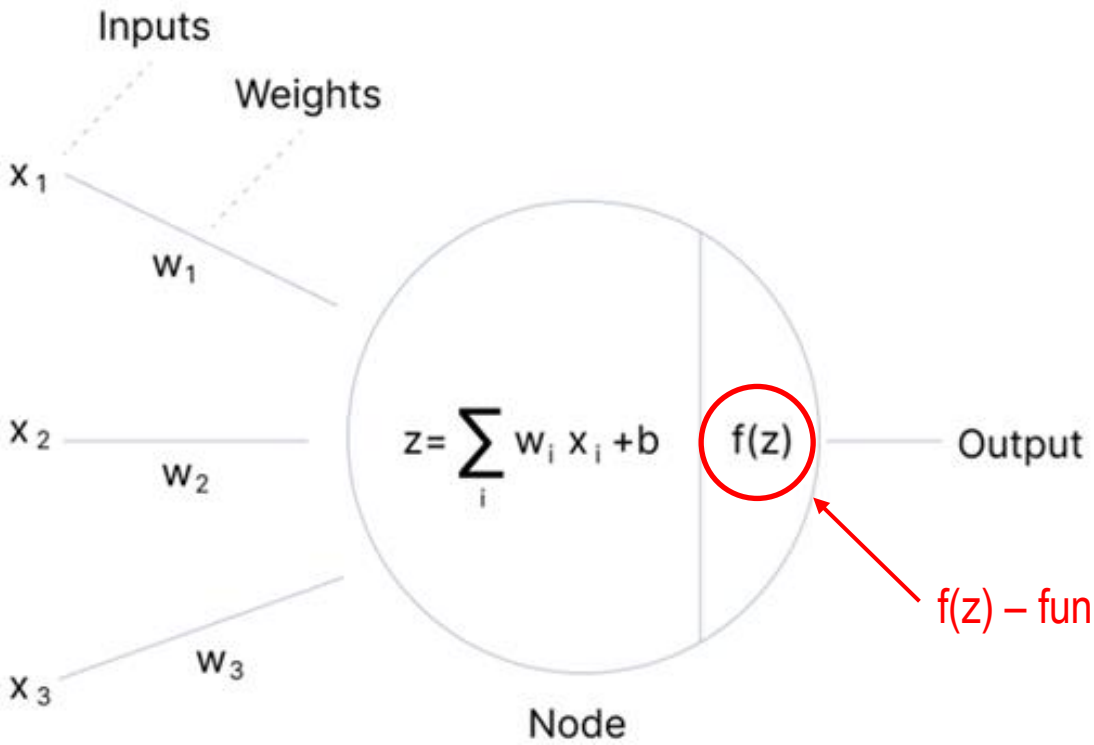
Adadelta automatycznie dopasowuje tempo uczenia w zależności od gradientu. W ten sposób sieć nie uczy się zbyt wolno i nie przeskakuje minimów z powodu zbyt dużych kroków. Optymalizacja Adadelta pozwala dynamicznie regulować parametry wraz z upływem czasu. Podobnie działają wspomniane **AdaGrad** i **RMSProp**.

Funkcje aktywacji

Funkcja aktywacji służy do obliczenia wartości wyjścia neuronów.
Pobiera ona wszystkie wejścia neuronu, oblicza jego wyjście i decyduje, czy przepuścić wartość wyjściową danego neuronu do kolejnej warstwy.

Zadaniem funkcji aktywacji jest wprowadzenie nieliniowości do sieci neuronowych. Bez tego sieci miałyby mniej skomplikowaną budowę, ale rozwiązanie za ich pomocą złożonych zadań byłoby niemożliwe. Modele bez funkcji aktywacji byłyby zwykłymi liniowymi modelami regresji.

Funkcja aktywacji pomaga sieci neuronowej wykorzystywać istotne informacje oraz tłumić mniej ważne.



$$w_1 * x_1$$
$$w_2 * x_2$$
$$w_n * x_n$$
$$z = \sum w_i * x_i + b$$

Różne typy neuronów (w naturze są ich tysiące) można zamodelować przy pomocy różnych funkcji aktywacji.

Funkcja aktywacji może przyjmować jedną z trzech postaci:

- liniowa,
- nieliniowa,
- skoku jednostkowego.

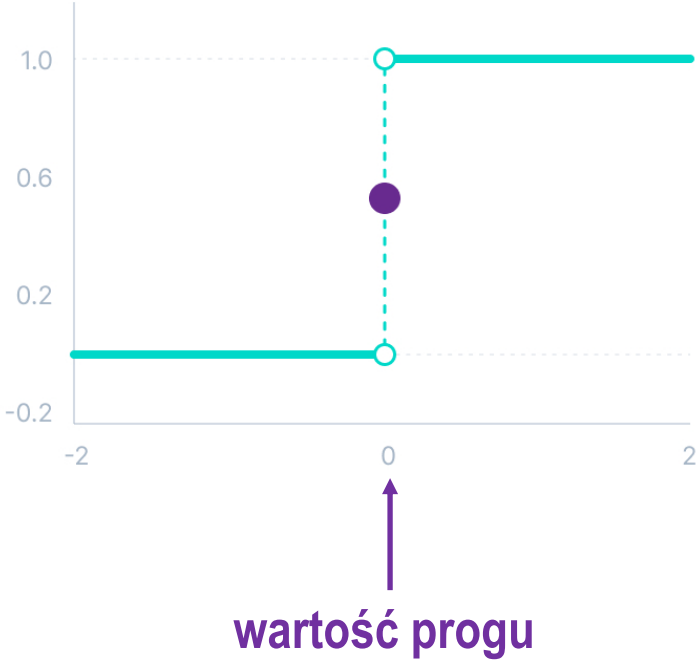
Wybór funkcji zależy od rodzaju problemu. Najczęściej stosowane są funkcje nieliniowe gdyż neurony o takich charakterystykach wykazują największe zdolności do nauki.

Zwykle neurony należące do tej samej warstwy mają tę samą funkcję aktywacji.

Funkcja progowa (ang. *binary step function*)

Przyjmuje wartości binarne (0 lub 1). W funkcji tej wybierana jest **wartość progu**.

- jeżeli wartość wejściowa znajduje się poniżej progu, neuron nie jest aktywowany (przyjmuje wartość 0, jego wyjście nie jest przekazywane do kolejnej warstwy ukrytej),
- jeżeli wartość wejściowa znajduje się powyżej progu, neuron jest aktywowany (przyjmuje wartość 1).



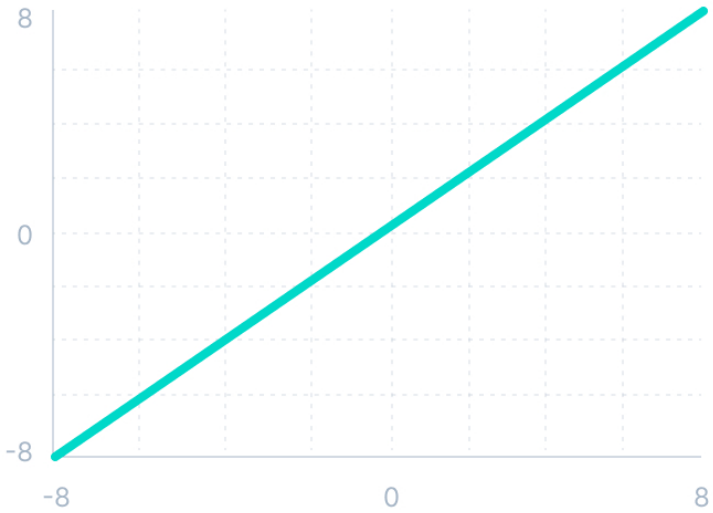
$$f(x) = \begin{cases} 0 & for\ x < 0 \\ 1 & for\ x \geqslant 0 \end{cases}$$

Funkcja progowa – własności:

- jest nieróżniczkowalna (jej gradient wynosi 0), więc przy jej użyciu nie można zastosować algorytmu propagacji wstecznej,
- sprawdza się w zadaniach klasyfikacji binarnej.

Funkcja liniowa, inaczej funkcja tożsamości (ang. *linear activation function, identity function*)

Wartość aktywacji jest proporcjonalna do wartości wejścia.



$$f(x) = x$$

Funkcja liniowa – własności:

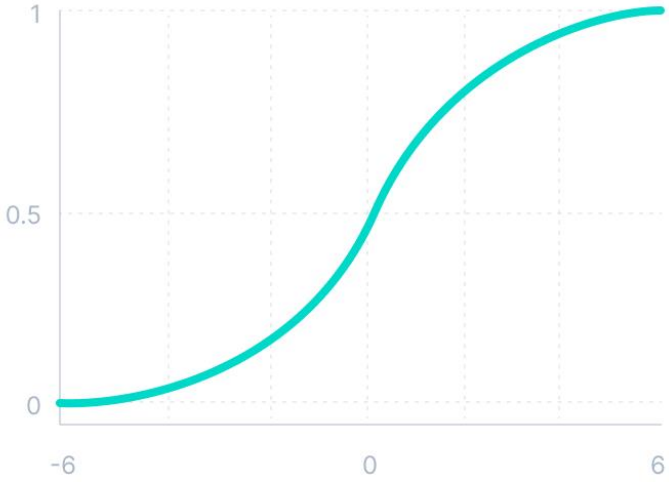
- pochodną funkcji liniowej jest wartość stała, która nie ma związku z wartością wejściową (x), w związku z tym przy użyciu tej funkcji nie można wykorzystać algorytmu propagacji wstecznej,
- zastosowanie tej funkcji spowoduje, że wszystkie warstwy sieci neuronowej zredukują się do jednej. Bez względu więc na liczbę warstw, ostatnia nadal będzie liniową funkcją pierwszej. Tak więc, liniowa funkcja aktywacji zamienia sieć neuronową w jedną warstwę.

Nieliniowe funkcje aktywacji

Nieliniowe funkcje aktywacji rozwiązują następujące ograniczenia funkcji liniowych:

- umożliwiają wykorzystanie algorytmu propagacji wstecznej, ponieważ ich pochodna jest związana z danymi wejściowymi i możliwe jest wskazanie, które wagi w neuronach wejściowych mogą zapewnić lepszą predykcję,
- umożliwiają tworzenie sieci wielowarstwowych, ponieważ ich wyjście jest nieliniową kombinacją danych wejściowych przepuszczonych przez wiele warstw. Każde wyjście może być reprezentowane jako wartość funkcji w sieci neuronowej.

Funkcja logistyczna, inaczej **sigmoidalna** – (ang. ***sigmoid/logistic activation function***)
Przekształca wartość wejściową w wartości z zakresu 0-1.



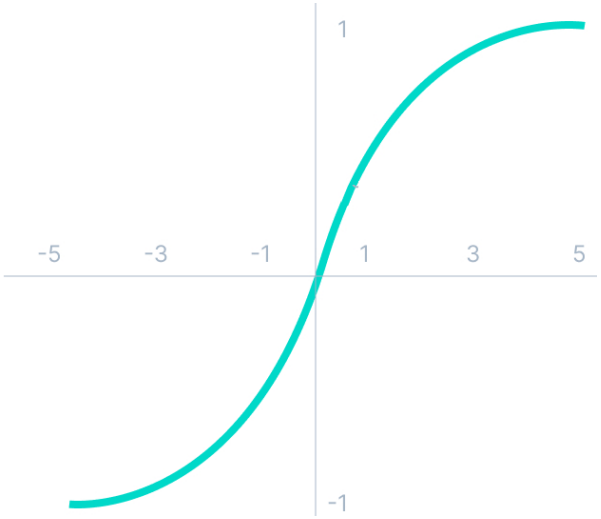
$$f(x) = \frac{1}{1 + e^{-x}}$$

Funkcja logistyczna – własności:

- jest jedną z najczęściej wykorzystywanych funkcji aktywacji,
- jest powszechnie używana w modelach, w których należy obliczyć prawdopodobieństwo jako wartość wyjściową, gdyż prawdopodobieństwo jest obliczane w przedziale od 0 do 1,
- jest różniczkowalna i daje płynną wartość gradientu, tzn. zapobiega skokom wartości wyjściowych. Jest to reprezentowane przez kształt litery S tej funkcji,
- wyjście nie jest symetryczne wokół zera, zatem wyjścia wszystkich neuronów będą miały ten sam znak. To sprawia, że trening sieci neuronowej jest trudniejszy i niestabilny,
- dobrze sprawdza się w zadaniach klasyfikacji.

Tangens hiperboliczny (ang. ***hyperbolic tangent***, **tanh**)

Przypomina funkcję sigmoidalną. Różni się od niej zakresem osi rzędnych (od –1 do 1), i jest od niej węższa. Dzięki temu ma większe pochodne, przez co minimalizuje błąd funkcji kosztu szybciej niż funkcja sigmoidalna.



$$f(x) = \frac{\left(e^{x} - e^{-x}\right)}{\left(e^{x} + e^{-x}\right)}$$

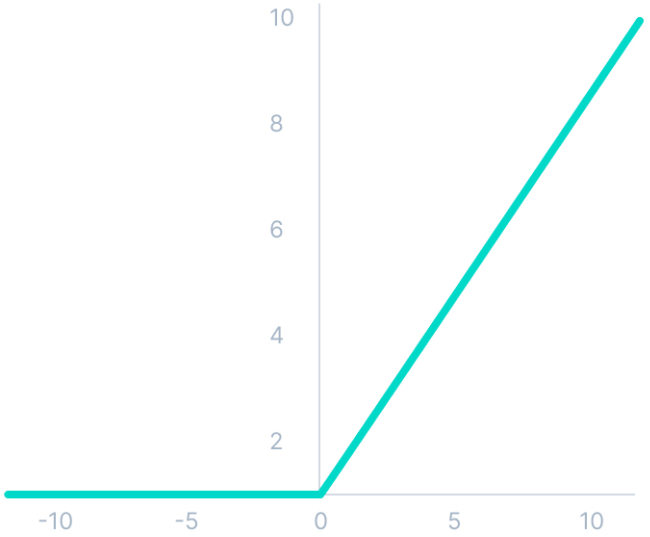
Tangens hiperboliczny – własności:

- wyjście jest symetryczne wokół zera, dzięki czemu można łatwo mapować wartości wyjściowe jako silnie negatywne, neutralne lub silnie pozytywne,
- często stosowana w warstwach ukrytych sieci neuronowej, ponieważ jej wartości mieszczą się w przedziale od –1 do 1, dlatego średnia dla warstwy ukrytej wynosi 0 lub jest bardzo do niej zbliżona. Pomaga to w centrowaniu danych i ułatwia uczenie dla następnej warstwy.

Rektyfikator (ang. ***Rectified Linear Unit*** – **ReLU**)

Przybiera wartość (0, x), dzięki czemu zeruje wartości ujemne.

Chociaż sprawia wrażenie funkcji liniowej, posiada funkcję pochodnej i pozwala na wsteczną propagację.

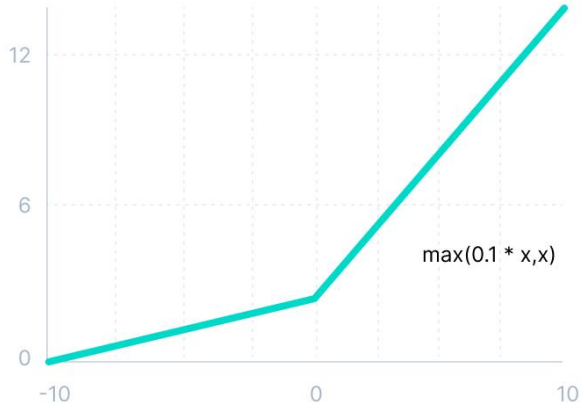


$$f(x) = max\,(0, x)$$

ReLU – własności:

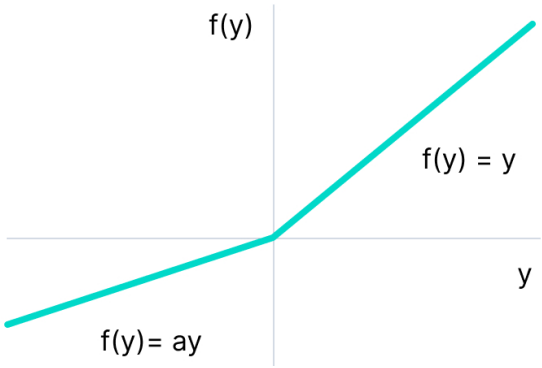
- ponieważ aktywuje tylko pewną liczbę neuronów, jest wydajniejsza obliczeniowo (szybsza) w porównaniu z funkcjami sigmoid i tanh,
- przyspiesza zejście gradientowe w kierunku globalnego minimum funkcji straty dzięki liniowej własności,
- podczas procesu wstecznej propagacji wagi dla niektórych neuronów nie są aktualizowane. Może to spowodować powstanie martwych neuronów, które nigdy nie zostaną uaktywnione,
- dobrze sprawdza się w zadaniach regresji.

Wersje ReLU:



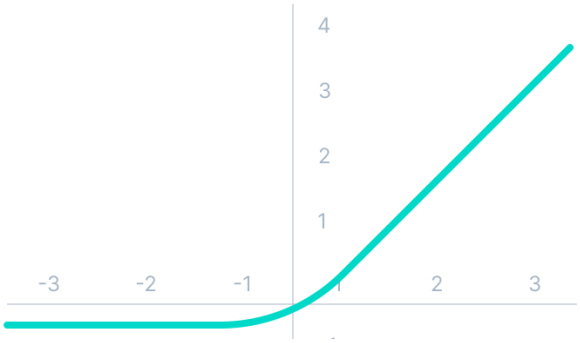
Leaky ReLU

Ulepszona wersja ReLU, rozwiązuje problem zanikających neuronów, ponieważ ma małe dodatnie nachylenie w obszarze ujemnym.



Parametric ReLU

Sparametryzowana funkcja ReLU jest używana, Leaky ReLU nie rozwiązuje problemu martwych neuronów, a istotne informacje nie są skutecznie przekazywane do następnej warstwy.



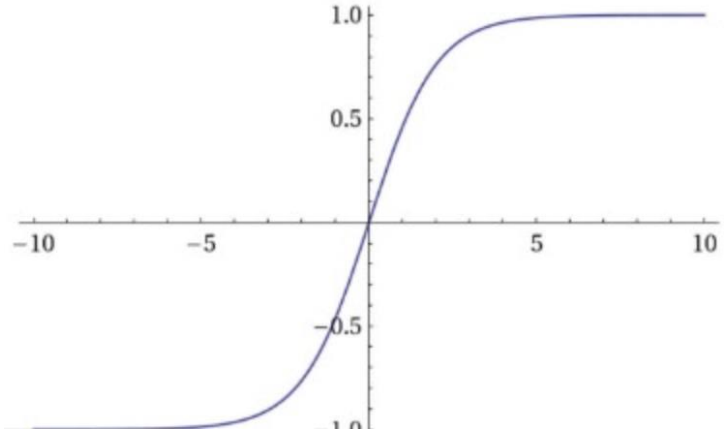
Exponential Linear Units (ELU)

Rozwiązuje problem zanikających neuronów występujący w ReLU poprzez wprowadzenie krzywej logarytmicznej dla ujemnych wartości danych wejściowych. Pomaga to sieci kierować wagi i skosy we właściwym kierunku.

Softmax

Jest kombinacją wielu sigmoidów.

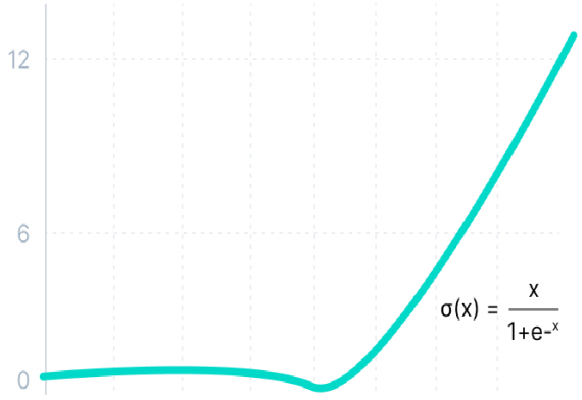
Jest najczęściej stosowana jako funkcja aktywacji dla ostatniej warstwy sieci neuronowej w przypadku klasyfikacji wieloklasowej.



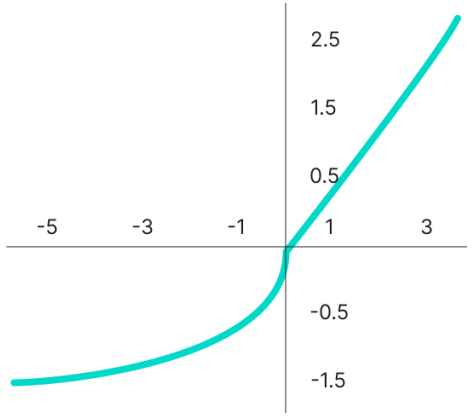
$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Inne funkcje aktywacji:

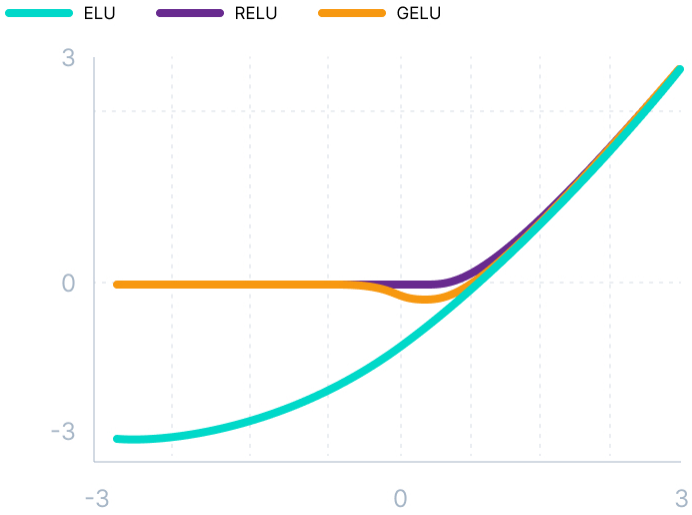
Swish



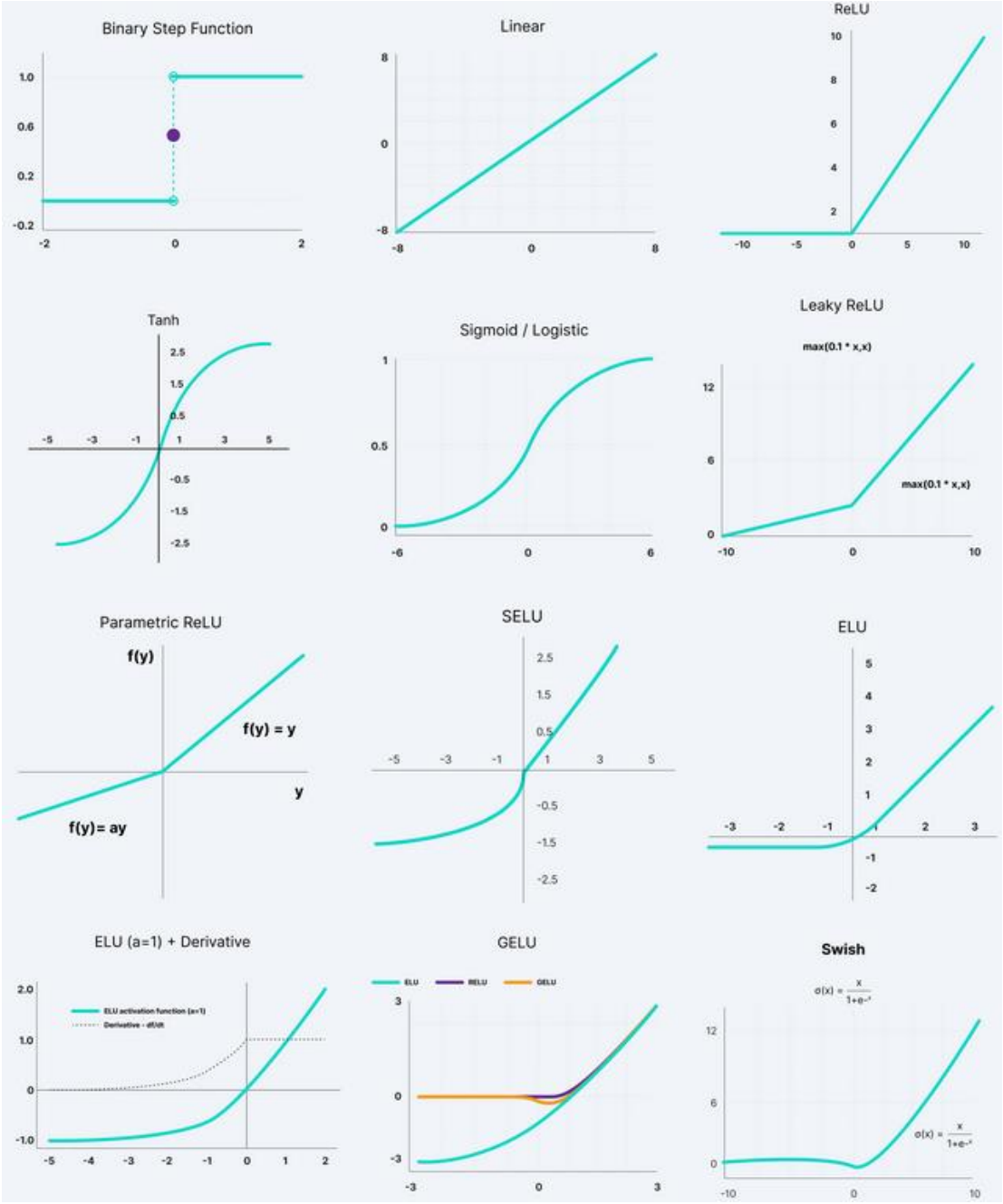
Scalled Exponential Linear Unit (SELU)



Gaussian Error Linear Unit (GELU)



Zestawienie wybranych funkcji aktywacji



https://www.v7labs.com/blog/neural-networks-activation-functions
https://ml-cheatsheet.readthedocs.io/en/latest/activation_functions.html

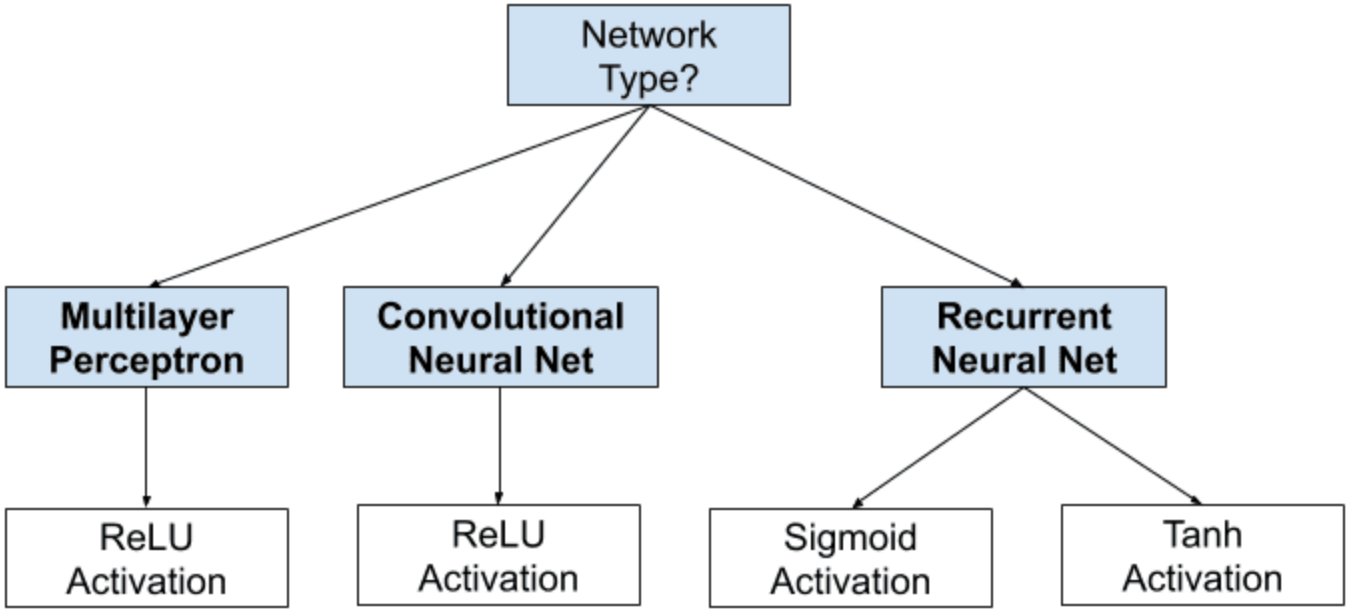
Wybór funkcji aktywacji w warstwach ukrytych:

- jedne funkcje aktywacji lepiej sprawdzają się w warstwach ukrytych, inne w warstwie wyjściowej,
- zwykle wykorzystuje się tę samą funkcję aktywacji we wszystkich warstwach ukrytych,
- funkcję aktywacji dobiera się do rodzaju problemu,
- funkcja aktywacji ReLU jest najczęściej używaną funkcją w warstwach ukrytych i nie powinna być używana w warstwie wyjściowej; jedna z zasad mówi, że należy zacząć od użycia w warstwach ukrytych funkcji aktywacji ReLU, a następnie próbować wybierać inne funkcje, jeśli ReLU nie zapewnia optymalnych wyników,
- funkcja Swish jest stosowana w sieciach neuronowych o głębokości powyżej 40 warstw,

Wybór funkcji aktywacji w warstwach ukrytych, c.d.:

- zależnie od struktury sieci, w **warstwach ukrytych** zazwyczaj należy stosować:

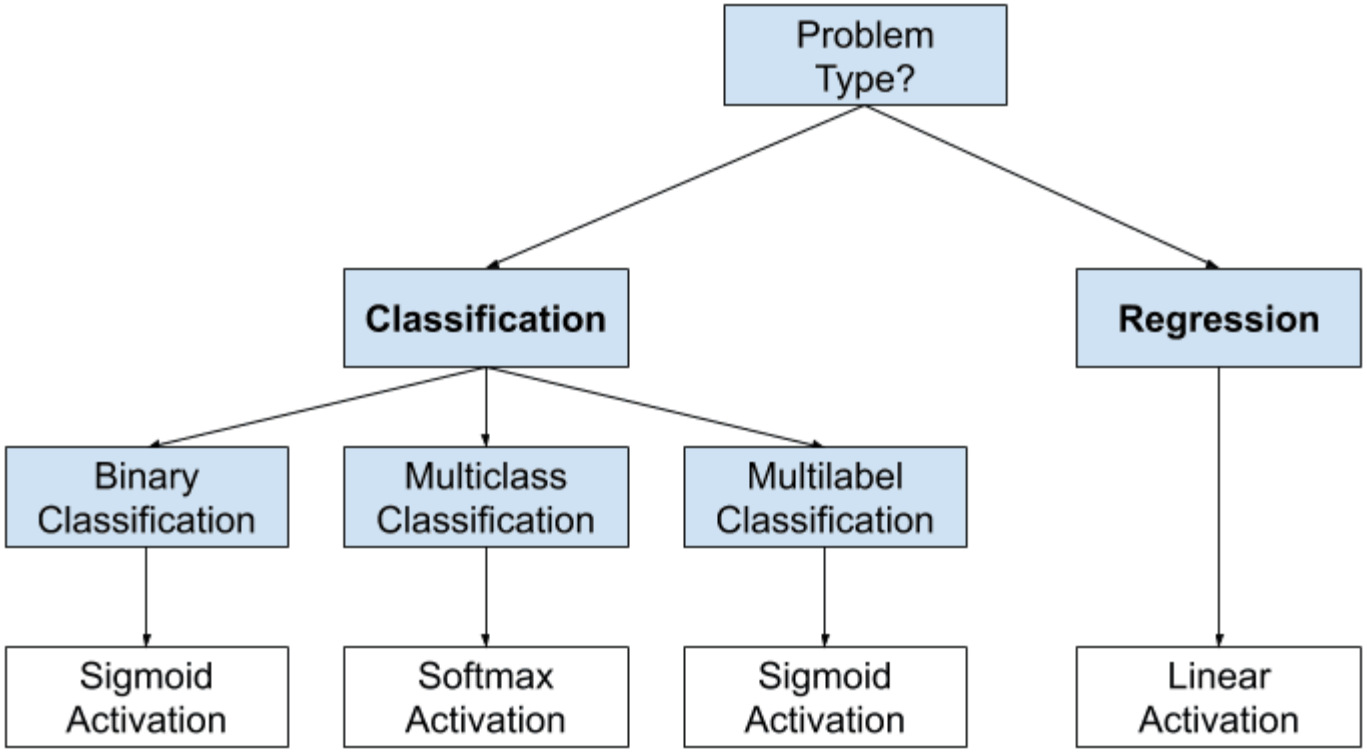
sieci konwolucyjne (CNN)	ReLU lub odmiany ReLU
sieci rekurencyjne (RNN)	Tanh lub Sigmoid
perceptrony wielowarstwowe (MLP)	ReLU lub odmiany ReLU



Wybór odpowiedniej funkcji aktywacji w warstwie wyjściowej:

- zależnie od rozwiązywanego problemu w **warstwie wyjściowej** powinno się stosować:

regresja	funkcja liniowa
klasyfikacja binarna (binary classification)	sigmoid
klasyfikacja wieloklasowa (multiclass classification)	softmax
klasyfikacja wieloetykietowa (multilabel classification)	sigmoid



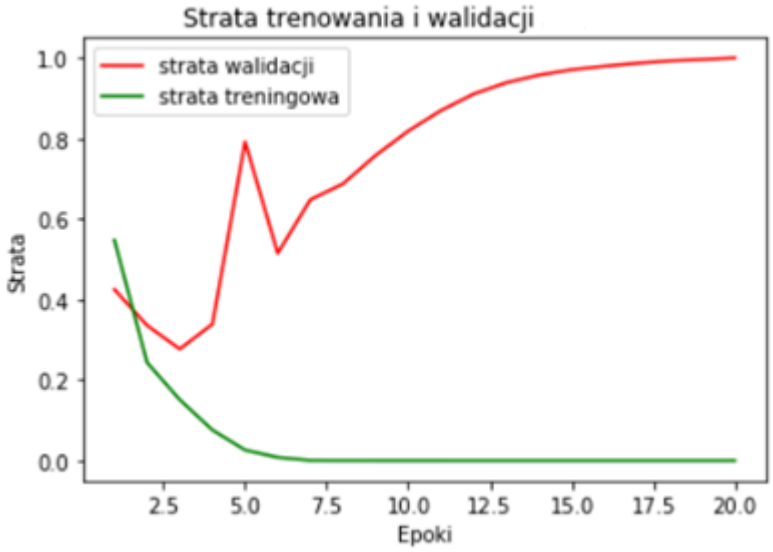
Funkcje aktywacji – podsumowanie:

- są wykorzystywane do wprowadzenia nieliniowości do sieci neuronowej,
- sieć neuronowa prawie zawsze będzie miała tę samą funkcję aktywacji we wszystkich warstwach ukrytych. Użyta funkcja aktywacji powinna być różniczkowalna, aby parametry sieci mogły być uczone z wykorzystaniem algorytmu wstecznej propagacji błędów,
- przy wyborze funkcji aktywacji należy brać pod uwagę problemy, z jakimi może się to wiązać, np. zanikające i eksplodujące gradienty,
- w przypadku warstwy wyjściowej należy zawsze brać pod uwagę zakres oczekiwanych wartości predykcji. Jeśli może to być dowolna wartość liczbowa (jak w przypadku problemu regresji), można użyć liniowej funkcji aktywacji lub ReLU,

Przeuczenie i regularyzacja

Istotnym problemem w głębokich sieciach neuronowych, szczególnie jeśli w danych występuje wiele parametrów, jest problem **przeuczenia** czyli nadmiernego dopasowywania się do danych treningowych.

Nadmierne dopasowanie (przetrenowanie, przeuczenie) modelu ma miejsce, gdy wydajność modelu podczas przetwarzania nowych danych zaczyna się pogarszać w porównaniu z wydajnością podczas przetwarzania zbioru treningowego. Nadmierne dopasowanie oznacza, że model idealnie pasuje do danych treningowych ale nie da się go uogólnić dla nowych danych pochodzących ze zbioru testowego. Jeśli dokładność modelu dla danych testowych nie jest zbliżona do dokładności dla danych treningowych, oznacza to, że model jest nadmiernie dopasowany.



Jeśli sieć dysponuje zbyt małą zdolnością zapamiętywania, to nie będzie się w stanie nauczyć bezpośredniego mapowania (zapamiętywania), a więc w celu minimalizacji strat będzie musiała nauczyć się przewidywać, a o to nam właśnie chodzi.

Jednocześnie modele powinny mieć na tyle dużo parametrów aby nie ulec zbyt słabemu dopasowaniu, model musi mieć możliwość zapamiętywania kolejnych cech.
Oznacza to, że trzeba znaleźć kompromis między zbyt dużą pojemnością a zbyt małą pojemnością.

Podstawowym problemem uczenia maszynowego jest więc kompromis między optymalizacją a uogólnianiem.

Optymalizacja to proces dostrajania modelu w celu uzyskania najlepszej możliwej wydajności na danych treningowych a **uogólnianie** pokazuje, jak dobrze zachowuje się model dla danych testowych, których nigdy nie przetwarzał. Uczenie maszynowe polega na dążeniu do jak najlepszej jakości uogólnienia ale nie mamy na to wpływu w sposób bezpośredni, gdyż podczas trenowania używane są tylko dane treningowe.

Przykład:

Na początku trenowania optymalizacja i uogólnianie są ze sobą skorelowane – strata dla danych treningowych i walidacyjnych maleje.

Po pewnej liczbie iteracji (epok) wyniki trenowania przestają ulegać poprawie a metryka walidacji przyjmuje wartość stałą lub się pogarsza – zaczyna pojawiać się nadmierne dopasowanie: model zaczyna uczyć się wzorców, które są specyficzne dla danych treningowych i nieprzydatne do przetwarzania nowych danych.



Celem uczenia maszynowego jest uzyskanie modelu zdolnego do jak najlepszych uogólnień a nie jak najlepiej dopasowanego do danych treningowych.

Proces walki z nadmiernym dopasowaniem nazywany jest **regularyzacją** sieci.
Regularyzacja to każdy mechanizm ułatwiający sieci poznanie istotnych cech rozkładu głównego a nie szczegółów zestawu danych.

Techniki zmniejszające niebezpieczeństwo przeuczenia sieci:

Wczesne zatrzymanie – polega na znalezieniu punktu, w którym błąd na próbie testowej jest jak najmniejszy. Jeśli proces uczenia zostanie skończony przed tym punktem – model będzie niedouczony. Jeśli zostanie zatrzymany po tym punkcie – model będzie przeuczony.

Sposobem na znalezienie tego punktu jest obserwacja działania algorytmu na dwóch zbiorach: treningowym i walidacyjnym. Jeżeli przy zmniejszaniu błędu na zbiorze treningowym, błąd na zbiorze walidacyjnym zaczyna rosnąć, należy przerwać uczenie sieci.



Techniki zmniejszające niebezpieczeństwo przeuczenia sieci, c.d.:

Redukcja rozmiaru sieci – polega na zmniejszeniu rozmiaru modelu, czyli liczby uczonych parametrów (liczba uczonych parametrów to tzw. **pojemność modelu**) – warstw i tworzących je jednostek (neuronów).

Ekspansja danych – dodatkowe dane w zbiorze treningowym mogą wpłynąć pozytywnie na jakość predykcji systemu. Zwiększenie liczby danych można uzyskać za pomocą następujących metod:

- zwiększenie zbioru danych w sposób konwencjonalny,
- dodanie szumu informacyjnego do posiadanych danych,
- przeprocesowanie i tym samym rozszerzenie zbioru danych (tzw. *augmentacja*, ang. *data augmentation*), np. w przypadku zdjęć może to być dodanie lustrzanego odbicia,
- tzw. **regularyzacja,** ang. ***regularization*** – modyfikacja algorytmu uczącego mająca na celu zmniejszenie błędu na próbie testowej (ale nie treningowej).

Regularyzacja:

Model zawierający kilka wag o dużych wartościach jest bardziej skomplikowany od modelu mającego mniejsze wagi. Utrzymywanie wag o małych wartościach powinno umożliwić sieci implementowanie prostszej funkcji i lepiej dopasowanej do wyznaczonego zadania.

Poprzez regularyzację minimalizuje się wagi wejść, które mają niewielki wpływ na końcową klasyfikację bądź też wybiera się tylko te neurony, które są użyteczne dla modelu. Jednakże, nie zawsze wiadomo, które wagi są bezużyteczne, więc minimalizujemy ich wpływ za pomocą modyfikacji funkcji straty.

W tym celu dodajemy do funkcji kosztu **regularyzator**. Implementuje się go poprzez dodanie do funkcji straty kosztu związanego z dużymi wartościami wag. Wtedy model staje się bardziej regularny.

Regularyzacja sieci, c.d.:

Istnieją trzy podstawowe metody regularyzacji:

L2 – pomaga utrzymywać małe wagi sieci. Dodaje do funkcji straty stały człon (tzw. koszt) proporcjonalnie do **kwadratu wartości** współczynników wag. Sieć, zamiast zerować nieznaczące wagi, daje im niższą wartość. Regularyzacja **L2** utrzymuje wagi na niskim poziomie ale pilnuje, aby nie były równe zero. Taka technika zapewnia, że żadne wejście nie zostanie pominięte.

L1 – pomaga utrzymywać małe wagi sieci. Koszt jest dodawany proporcjonalnie do **wartości bezwzględnej** współczynników wag. Rzadziej stosowana niż L2.

Dropout – polega na losowym usuwaniu neuronów (wraz z ich połączeniami) podczas treningu. Parametrem dropout jest liczba rzeczywista, np.: Dropout(0.25) oznacza, że każdy neuron jest losowo odrzucany raz na cztery cykle. Wyboru neuronów do usunięcia dokonuje się w sposób losowy, z ustalonym z góry prawdopodobieństwem dla każdego neuronu. Takie podejście zmusza sieć do mniejszego polegania na pojedynczych neuronach, ponieważ mogą one zostać nieuwzględnione w strukturze sieci podczas dowolnej iteracji.
Z doświadczeń wynika, że Dropout poprawia parametry modeli uczenia nadzorowanego wykorzystywanych do rozpoznawania obrazów, mowy, klasyfikacji tekstu. Dropout wprowadzamy bezpośrednio przed warstwą spłaszczoną (Flatten). Aby zwiększyć dokładność modelu można dodać więcej warstw Dropout za warstwą ukrytą.

Regularyzacja sieci, c.d.:

Batch Normalization – normalizuje aktywację warstwy we wsadzie, np. wykonuje transformację, która utrzymuje średnią aktywację wokół 0 z odchyleniem standardowym bliskim 1. Technika ta także pełni rolę optymalizatora, w pewnych przypadkach eliminując potrzebę używania Dropout. Użycie Batch Normalization pozwala na uzyskanie porównywalnej dokładności trenowania przy mniejszej liczbie kroków oraz przyspiesza sam proces trenowania.

Metody ewaluacji modeli

Metody **ewaluacji** (walidacji) modeli, czyli miary wydajności:

1. Walidacja na odłożonych danych (ang. *hold-out data*)

2. Dokładność trenowania (ang. *accuracy*)

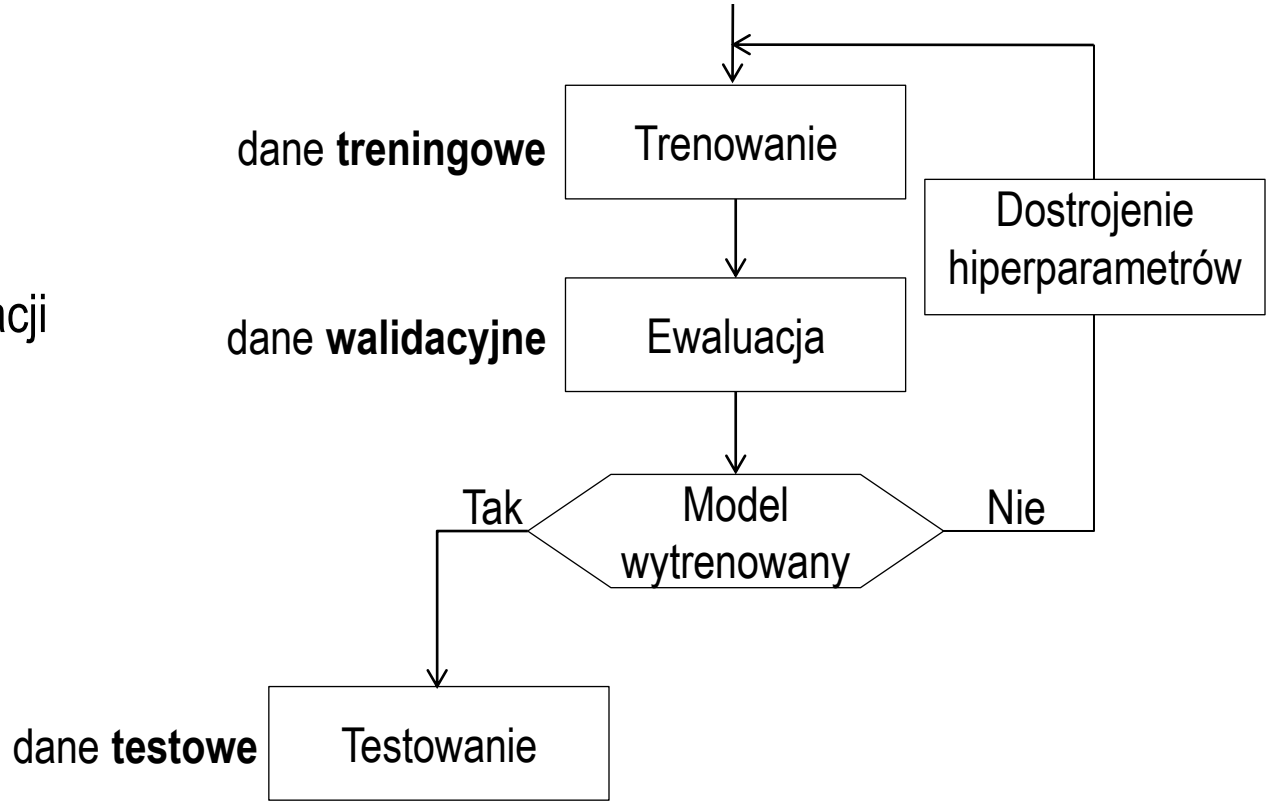
3. Walidacja krzyżowa:

k-krotna (klasyczna),
k-krotna z losowaniem,
stratyfikowana,

4. Macierz pomyłek

Walidacja na odłożonych danych:

1. Trenowanie modelu na danych **treningowych**
2. Ewaluacja modelu na danych **walidacyjnych**
3. Dostrajanie hiperparametrów na podstawie wyników ewaluacji i powrót do punktu 1. Procedura jest powtarzana aż model uzyska satysfakcjonujące parametry
4. Ocena jakości modelu na danych **testowych** (**odłożonych**)



Wady metody walidacji na odłożonych danych:

- jeśli podczas losowania różnych próbek do poszczególnych zbiorów uzyskujemy różną wydajność modelu, oznacza to, że mamy zbyt mało próbek i zbiór danych nie jest reprezentatywny,
- wydajność modelu może być w dużej mierze zależna od tego, które obserwacje zostały wybrane do zbioru testowego,
- model nie jest trenowany ani oceniany z użyciem wszystkich dostępnych danych.

Dokładność trenowania (accuracy):

$$ACC = \frac{Np}{N}$$

gdzie:
Np – liczba przykładów testowych sklasyfikowanych prawidłowo przez model
N – liczba wszystkich przykładów testowych

Nie jest to dobra miara w przypadku zbiorów danych, w których niektóre klasy występują częściej od pozostałych.
Np.:
gdybyśmy mieli zbiór zdjęć, na których byłoby 90% kotów i 10% psów, to nawet, gdybyśmy zgadywali, czy na zdjęciu jest kot, to w 90% przypadków mielibyśmy rację.
Jeśli wytrenowany model zwracałby zawsze informację, że na zdjęciu jest kot, to jego dokładność byłaby równa 90%.

Dokładność trenowania (accuracy), c.d.:

Jeśli, w przypadku klasyfikacji binarnej, przyjmiemy:
N_0 – liczbę wystąpień klasy 0 w zbiorze testowym
N_1 – liczbę wystąpień klasy 1 w zbiorze testowym
C_0 – liczbę poprawnie wytypowanych wystąpień klasy 0 przez model
C_1 – liczbę poprawnie wytypowanych wystąpień klasy 1 przez model

to możemy policzyć dokładność dla każdej klasy:

$$ACC_0 = \frac{C_0}{N_0} \qquad ACC_1 = \frac{C_1}{N_1}$$

oraz dokładność ogólną:

$$ACC = \frac{C_0 + C_1}{N_0 + N_1} = \frac{Np}{N}$$

Dokładność trenowania (accuracy), c.d.:

$$ACC_0 = \frac{C_0}{N_0} \qquad ACC_1 = \frac{C_1}{N_1} \qquad ACC = \frac{C_0 + C_1}{N_0 + N_1} = \frac{Np}{N}$$

Przykład:
w zbiorze zdjęć jest 90% zdjęć kotów (etykieta 0) i 10% zdjęć psów (etykieta 1).
Model klasyfikuje wszystkie zdjęcia jako koty.

Wtedy:

ACC = 0.9

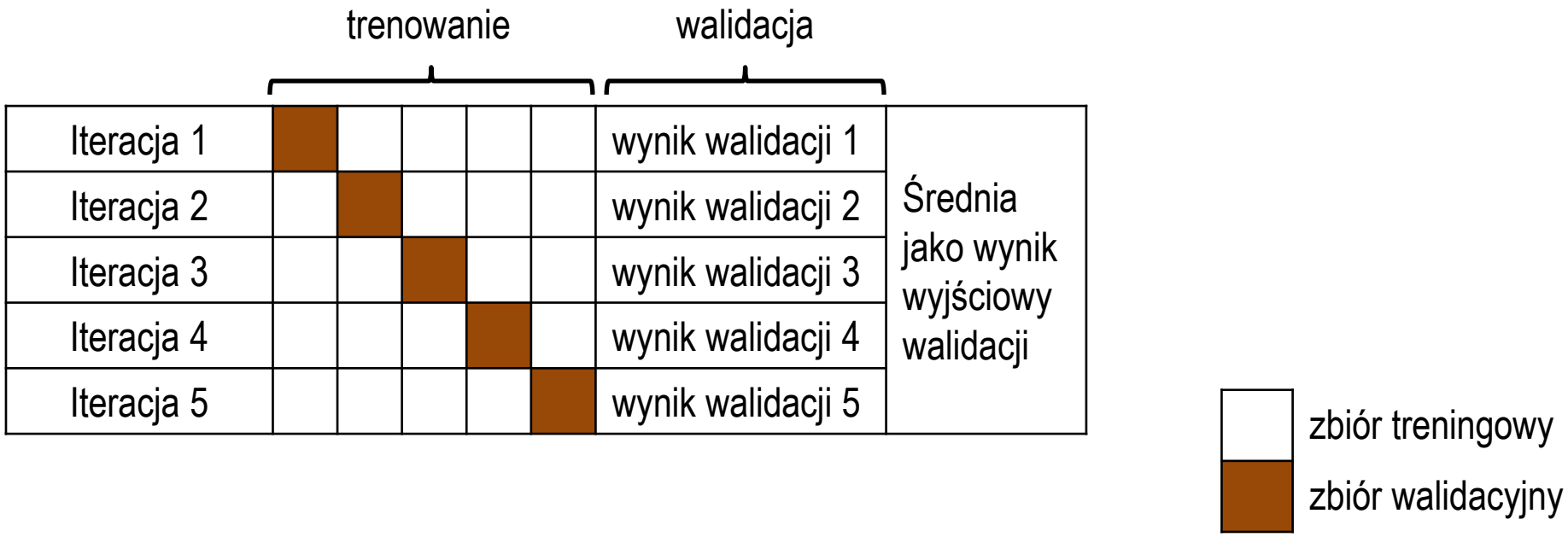
$ACC_0 = 1$

$ACC_1 = 0$

Klasyczna walidacja krzyżowa:

Wykorzystanie zbioru treningowego i walidacyjnego w procesie uczenia na przykładzie **k-krotnej walidacji krzyżowej** (ang. *k-fold cross-validation*, KFCV):

1. Dane są dzielone na **k**-równych części (podzbiorów)
2. Dla każdej części **i** model jest trenowany na pozostałych częściach i walidowany na części **i**
3. Ostatecznym wynikiem jest średnia **k** wyników



K-krotna walidacja z losowaniem

polega na wielokrotnym stosowaniu k-krotnej walidacji krzyżowej, w której przed każdym kolejnym etapem próbki do poszczególnych zbiorów są losowane. Ostateczny wynik walidacji jest średnią wyników uzyskanych podczas każdego etapu.
W metodzie tej trenowanych i ewaluowanych jest **p*k** modeli, gdzie **p** jest liczbą iteracji algorytmu.

Stratyfikowana walidacja krzyżowa (ang. ***stratified cross-validation***)

polega na takim podziale obiektów pomiędzy poszczególne podzbiory, aby zachowane były proporcje pomiędzy klasami decyzyjnymi. Stosowana, gdy w danych występują znaczne dysproporcje w liczebności przykładów należących do poszczególnych klas (np. w zbiorze zdjęć jest 80% motocykli i 20% rowerów).

Macierz pomyłek

(tablica kontyngencji, macierz błędów, ang. *confusion matrix*):

1. **Poprawne zidentyfikowanie** wartości pozytywnej (true-positive) lub wartości negatywnej (true-negative).
   Np.:
   osoba zdrowa została poprawnie rozpoznana jako zdrowa (true-positive)
   osoba chora została poprawnie rozpoznana jako chora (true-negative)
2. **Błędne zidentyfikowanie** wartości pozytywnej jako negatywna (false-negative) lub wartości negatywnej jako pozytywna (false-positive).
   Np.:
   osoba zdrowa została błędnie rozpoznana jako chora (false-negative)
   osoba chora została błędnie rozpoznana jako zdrowa (false-positive)

		Wartość przewidywana	
		T(1)	F(0)
Wartość rzeczywista	T(1)	true-positive	false-negative
	F(0)	false-positive	true-negative

		Wartość przewidywana	
		T(1)	F(0)
Wartość rzeczywista	T(1)	true-positive (TP)	false-negative (FN)
	F(0)	false-positive (FP)	true-negative (TN)

Miary poprawności predykcji:

Precyzja (ang. *precision*) – jaka część przewidywań pozytywnych jest faktycznie pozytywna:

$$\frac{TP}{TP+FP}$$

Czułość (pełność, ang. *recall*) – odsetek prawidłowo sklasyfikowanych obserwacji pozytywnych:

$$\frac{TP}{TP+FN}$$

Miary te są wykorzystywane w zależnie od sytuacji.
Przykład 1: jeśli klasyfikujemy filmy odpowiednie dla dzieci, zależy nam na tym, aby…
FP było jak najmniejsze, czyli precyzja jak największa (aby zminimalizować liczbę filmów zakwalifikowanych błędnie jako odpowiednie dla dzieci), nawet kosztem dużego FN (odrzucenie dużej liczby filmów odpowiednich).

		Wartość przewidywana	
		T(1)	F(0)
Wartość rzeczywista	T(1)	true-positive (TP)	false-negative (FN)
	F(0)	false-positive (FP)	true-negative (TN)

Miary poprawności predykcji:

Precyzja (ang. *precision*) – jaka część przewidywań pozytywnych jest faktycznie pozytywna:

$$\frac{TP}{TP+FP}$$

Czułość (pełność, ang. *recall*) – odsetek prawidłowo sklasyfikowanych obserwacji pozytywnych:

$$\frac{TP}{TP+FN}$$

Przykład 2: jeśli rozpoznajemy złodziei z zapisów kamer, zależy nam aby…
FN było jak najmniejsze, czyli czułość jak największa (liczba wykrytych złodziei będzie wtedy duża), nawet kosztem dużego FP (co wygeneruje dużą liczbę nieprawidłowych alarmów poprzez błędne rozpoznanie osoby jako złodzieja).

		Wartość przewidywana	
		T(1)	F(0)
Wartość rzeczywista	T(1)	true-positive (TP)	false-positive (FP)
	F(0)	false-negative (FN)	true-negative (TN)

F_1 – średnia harmoniczna precyzji i czułości. Wstażnik ten jest wykorzystywany do porównania obydwu klasyfikatorów. F_1 będzie miał dużą wartość, gdy zarówno precyzja jak i czułość będą miały dużą wartość:

$$F1 = \frac{2}{\frac{1}{Precyzja} + \frac{1}{Czułość}}$$

		Wartość przewidywana	
		T(1)	F(0)
Wartość rzeczywista	T(1)	true-positive (TP)	false-positive (FP)
	F(0)	false-negative (FN)	true-negative (TN)

Inne miary poprawności predykcji, c.d.:

Dokładność (trafność, *accuracy*) – zlicza odsetek poprawnych przewidywań klasyfikatora:

$$\frac{TP+TN}{TP+TN+FP+FN}$$

Specyficzność – odsetek prawidłowo sklasyfikowanych obserwacji negatywnych:

$$\frac{TN}{FP+TN}$$

Przykład:

Spośród 30 przebadanych osób, 24 zostały zakwalifikowane jako zdrowe a 6 jako chore.

Po dokładnym badaniu okazało się, że wśród 24 zakwalifikowanych jako zdrowe 2 okazały się chore a wśród 6 zakwalifikowanych jako chore 1 okazała się zdrowa. Wypełnij macierz pomyłek.

		Wartość przewidywana		
		Z	Ch	Suma
Wartość rzeczywista	Z	22 (TP)	1 (FN)	23
	Ch	2 (FP)	5 (TN)	7
Suma		24	6	

Przykład, c.d.:
Oblicz miary poprawności predykcji.
Odpowiedz na pytania: Ile osób zostało błędnie sklasyfikowanych?
Która sytuacja jest najbardziej niebezpieczna i dlaczego?

		Wartość przewidywana	
		Z	Ch
Wartość rzeczywista	Z	22 (TP)	**1 (FN)**
	Ch	**2 (FP)**	5 (TN)

Precyzja (*precision*):

$$\frac{TP}{TP+FP} \qquad \frac{22}{22+2} = 0{,}92$$

Czułość (*recall*):

$$\frac{TP}{TP+FN} \qquad \frac{22}{22+1} = 0{,}96$$

Trafność (*accuracy*):

$$\frac{TP+TN}{TP+TN+FP+FN} \qquad \frac{22+5}{22+5+2+1} = 0{,}90$$

Odpowiedzi:
Błędnie sklasyfikowane zostały trzy osoby.
Najlepsza okazała się miara czułości.

W tym zadaniu najbardziej niebezpieczna jest sytuacja zakwalifikowania osoby chorej jako zdrowej, więc należałoby dążyć do maksymalizacji miary precyzji.

Budując model należy podjąć szereg decyzji:

1. **Jaką liczbę warstw i neuronów wybrać?**

   zbyt mała liczba neuronów – sieć nie będzie się w stanie nauczyć

   zbyt duża liczba neuronów – wydłużenie czasu uczenia i tendencja do przeuczania

   - metoda piramidy (podejście przecinające, ang. *pruning approach*) – zaczynamy od sieci przewymiarowanej i usuwamy kolejno mniej ważne neurony
   - podejście konstruktywne (ang. *constructive approach*) – uczenie rozpoczyna się z małą liczbą warstw i neuronów i zwiększa się ich liczbę obserwując parametry modelu

   Obie metody są, w pewnym stopniu, metodami prób i błędów. Nie ma wypracowanej metody, za pomocą której można określić liczbę neuronów w warstwie.
2. **Jaką zastosować formę nauczania (nadzorowane, nienadzorowane, ze wzmocnieniem)?**
3. **Jaką wybrać strukturę modelu (jednokierunkowy, rekurencyjny, sieć Kohonena, maszyna Boltzmanna, graf acykliczny)?**
4. **Jak duży wsad (batch) zastosować?**
5. **Jaką funkcję celu wybrać?**
6. **Jaką wybrać funkcję aktywacji?**
7. **Jaki zastosować optymalizator?**
8. **Czy korzystać z regularyzacji, jeśli tak to jakiej i kiedy?**
9. **Jaką wybrać metodę ewaluacji?**

Wybór hiperparametrów modelu

	Klasyfikacja	Regresja
Funkcja straty	• Entropia krzyżowa (binary, categorical lub sparse) • Logarytmiczna • Wykładnicza	• Średni błąd kwadratowy (mse) • Średni błąd bezwzględny (mae) • Funkcja Hubera (odmiana mae) • Logarytmiczny cosinus hiperboliczny
Algorytm	• Drzewo decyzyjne • Metoda k-średnich • Met. wektorów wspierających (SVM)	• Metoda wektorów wspierających (SVM)
Funkcja aktywacji	• Sigmoid (zwykle w ostatniej warstwie) • Softmax • Rektyfikator (ReLu) • elu • Funkcja progowa (klasyfikacja binarna) • Tangens hiperboliczny (tanh)	• Rektyfikator (ReLu)

Wybór funkcji aktywacji ostatniej warstwy i funkcji straty modelu

Rodzaj zadania	Funkcja aktywacji ostatniej warstwy	Funkcja straty
Klasyfikacja binarna	sigmoid	binary_crossentropy
Klasyfikacja wieloklasowa	softmax	categorical_crossentropy
Klasyfikacja wieloetykietowa	sigmoid	binary_crossentropy
Regresja wartości z dowolnego zakresu	funkcja liniowa	mse
Regresja wartości z zakresu 0–1	sigmoid	mse binary_crossentropy

Uczenie maszynowe to **kompromis między optymalizacją a uogólnieniem**, między nadmiernym dopasowaniem a niedopasowaniem, a więc wymaga poszukiwania optymalnej pojemności modelu.

Eksperymentując należy zawsze obserwować wartości straty trenowania i walidacji. Spadek wydajności modelu na danych walidacyjnych oznacza, że osiągnięto nadmierne dopasowanie. Regularyzacja i dostrajanie pozwalają uzyskać model optymalny. Wymaga to wielokrotnego wykonywania następujących operacji:

- modyfikowanie modelu,
- trenowanie,
- ocena na walidacyjnym zbiorze danych.

Podczas takiego dostrajania modelu korzysta się z:

- mechanizmu porzucania,
- modyfikacji architektury sieci poprzez dodawanie i usuwanie warstw,
- dodawania regularyzacji L1 lub L2,
- definiowania i modyfikacji różnych hiperparametrów, jak liczba jednostek w poszczególnych warstwach, współczynnik uczenia się optymalizatora,
- prowadzenia inżynierii cech: dodawanie nowych, usuwanie tych, które wydają się nie zawierać przydatnych informacji.

W jaki sposób dobrać parametry sieci aby działała jak najlepiej?

„Dobór właściwej architektury sieci to raczej kwestia doświadczenia niż analizy naukowej. Jedyną odpowiedzią na to pytanie jest wykonanie wielu eksperymentów i prób. Może się okazać, że musimy nauczyć około 100 warstw sieci neuronowych, zanim osiągniemy optymalne parametry i strukturę dla zadania, które mamy rozwiązać. **Uczenie maszynowe to ciągłe uczenie się na błędach**”.

„Algorytmy uczenia maszynowego nie prowadzą do dokładnego rozwiązania problemu matematycznego a jedynie przybliżonego”.

Sieci neuronowe nie sprawdzają się w sytuacji, w której szukamy interpretacji wyniku, np. istotności cechy, która tłumaczy, w jakim stopniu dana cecha oddziałuje na zmienną objaśnianą.

W jaki sposób dobrać parametry sieci aby działała jak najlepiej, c.d.:

„Osoba zajmująca się uczeniem głębokim musi wypracować sobie intuicję na temat struktury głębokich sieci. Biorąc pod uwagę słaby stan teorii, praca empiryczna jest najlepszym sposobem nauki budowania modeli uczenia głębokiego”.

„Bądź systematyczny: zapisuj swoje wybory i wyniki w arkuszu kalkulacyjnym i systematycznie badaj teren. Spróbuj zrozumieć efekty działania różnych hiperparametrów”.